



# SW개발/HW제작 설계서

\* 프로젝트 명 : CBDC 기반 자율 주행 자동차 결제 지갑 구축

2025. 10. 30

MotoWallet

# 수행 단계별 주요 산출물

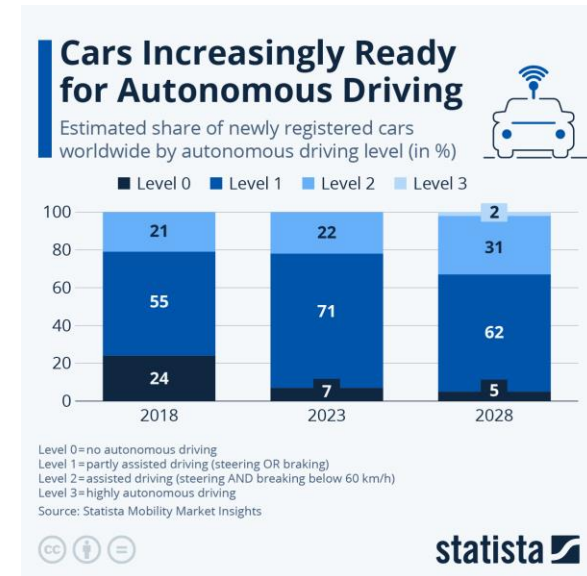
※ ○ 필수, △ 선택

| 단계      | 산출물              | 일반                 | 응용 소프트웨어                  | 응용 하드웨어              |
|---------|------------------|--------------------|---------------------------|----------------------|
|         |                  | ·모바일 APP<br>·Web 등 | ·빅데이터<br>·인공지능<br>·블록체인 등 | ·IoT<br>·로봇<br>·드론 등 |
| 환경 분석   | 시장/기술 환경 분석서     | △                  | △                         | △                    |
|         | 설문조사 결과서         | △                  | △                         | △                    |
|         | 인터뷰 결과서          | △                  | △                         | △                    |
| 요구사항 분석 | 요구사항 정의서         | ○                  | ○                         | ○                    |
|         | 유즈케이스 정의서        | ○                  | △                         | △                    |
| 아키텍처 설계 | 서비스 구성도(시스템 구성도) | ○                  | ○                         | ○                    |
|         | 서비스 흐름도(데이터 흐름도) | ○                  | ○                         | △                    |
|         | UI/UX 정의서        | ○                  | ○                         | △                    |
|         | 하드웨어/센서 구성도      | -                  | -                         | ○                    |
| 기능 설계   | 메뉴 구성도           | ○                  | ○                         | ○                    |
|         | 화면 설계서           | ○                  | ○                         | ○                    |
|         | 엔티티 관계도          | ○                  | ○                         | ○                    |
|         | 기능 처리도(기능 흐름도)   | ○                  | ○                         | ○                    |
|         | 알고리즘 명세서/설명서     | △                  | ○                         | ○                    |
|         | 데이터 수집처리 정의서     | -                  | ○                         | -                    |
|         | 하드웨어 설계도(완료)     | -                  | -                         | ○                    |
| 개발 / 구현 | 프로그램 목록          | ○                  | ○                         | ○                    |
|         | 테이블 정의서          | ○                  | ○                         | △                    |
|         | 핵심 소스코드          | ○                  | ○                         | ○                    |

## | 시장/기술 동향 분석

| 레벨 구분  | Level 0                  | Level 1                       | Level 2                        | Level 3                             | Level 4                     | Level 5                     |
|--------|--------------------------|-------------------------------|--------------------------------|-------------------------------------|-----------------------------|-----------------------------|
|        |                          |                               |                                |                                     |                             |                             |
|        | 운전자 보조 기능                |                               |                                | 자율주행 기능                             |                             |                             |
| 명칭     | 無자율주행<br>(No Automation) | 운전자 지원<br>(Driver Assistance) | 부분 자동화<br>(Partial Automation) | 조건부 자동화<br>(Conditional Automation) | 고도 자동화<br>(High Automation) | 완전 자동화<br>(Full Automation) |
| 자동화 항목 | 없음(경고 등)                 | 조향 or 속도                      | 조향 & 속도                        | 조향 & 속도                             | 조향 & 속도                     | 조향 & 속도                     |
| 운전주식   | 항시 필수                    | 항시 필수                         | 항시 필수                          | 시스템 요청시                             | 작동구간내 불필요                   | 전 구간 불필요                    |
| 자동화 구간 | -                        | 특정구간                          | 특정구간                           | 특정구간                                | 특정구간                        | 전 구간                        |
| 예시     | 사각지대 경고                  | 차선유지 또는 크루즈 기능                | 차선유지 및 크루즈 기능                  | 혼잡구간 주행지원 시스템                       | 지역(Local) 무인택시              | 운전자 없는 완전자율주행               |

<표 1> 운전 자동화의 단계적 구분



<표 2> 자율주행 기술의 발전

- 자율주행 기술의 발전은 전 세계 모빌리티 분야에 혁신적인 변화를 가져오고 있다.
- 자율주행차는 운전자의 개입 없이 스스로 주행할 수 있어 교통 효율성을 높이고, 사고를 줄이며, 교통 체증을 완화하는 등의 장점을 제공하고 있다.
- 자율주행 기술의 발전은 빠르게 진행되어 우리의 삶에 들어오고 있다.

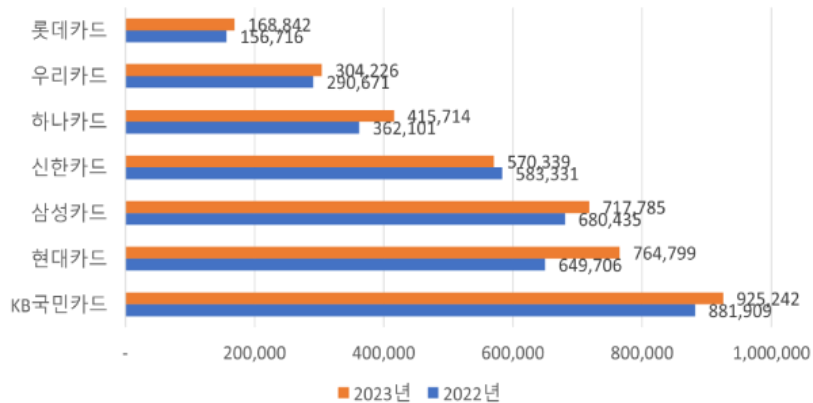
자료: 산업통상자원부(2021) ※ SAE 구분에 따른 분류,  
Cars Increasingly Ready for Autonomous Driving, statista

## | 시장/기술 동향 분석

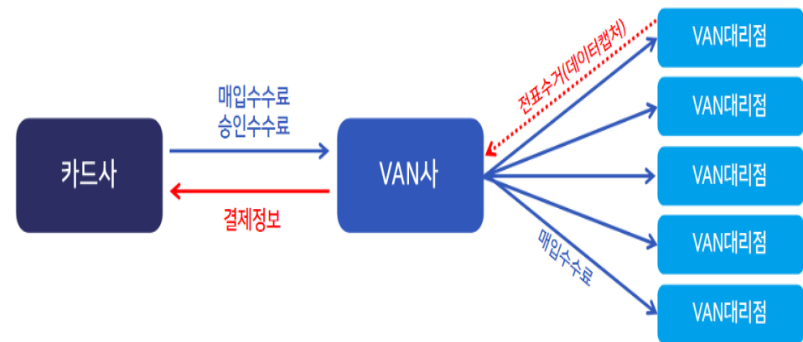
카드사 2022-2023년 3분기

가맹점수수료 수익 추이

[단위]: 백만원 [자료] 금융감독원 금융통계정보시스템



[ 2022~23 카드사 가맹점수수료 추이]



[카드사 VAN사 수수료 체계]

- 자율주행 기술의 발전과 함께 모빌리티 분야에서는 자동 처리할 수 있는 비접촉·실시간 결제 시스템이 필수적으로 요구된다.
- 기존 신용카드·모바일 결제는 **처리 지연**과 **수수료 문제**로 한계가 있다.
- 이 문제를 해결하기 위해 **중앙은행 디지털화폐(CBDC) 기반 결제 시스템**이 주목받고 있다.

# | 요구사항 정의서

| 구분      | 요구사항 ID       | 요구사항명          | 기능 ID       | 기능명     | 세부사항                                                                                                   |
|---------|---------------|----------------|-------------|---------|--------------------------------------------------------------------------------------------------------|
| S/W (앱) | Auth_User_001 | 회원가입           | AUTH001-001 | 인증 기능   | 최소한의 개인정보 수집을 위한, 인증 가능한 이메일 주소, 이름, ID 정보 입력, ID의 경우는 자동 생성 해준다.                                      |
|         | Auth_User_002 | 로그인            | AUTH001-002 | 인증 기능   | 이메일 및 패스워드 기반 로그인 사용자가 자동 로그인 체크 선택에 따라, 이메일 주소 저장                                                     |
|         | Home_User_001 | 지갑 요약 표시       | HOME001-001 | 홈 화면 기능 | 사용자 이름, 지갑 잔액, 지갑 주소 표시<br>* 지갑 주소 : 블록체인 발급 (26~35자리)<br>예)0x71C7656EC7ab88b098c56a197713698820391816 |
|         | Home_User_002 | 송금하기           | HOME001-002 | 홈 화면 기능 | Wallet_User_003                                                                                        |
|         | Home_User_003 | 최근 결제 내역 (더보기) | HOME001-003 | 홈 화면 기능 | Wallet_User_005-001-002<br>상위 세건 리스트업                                                                  |
|         | Home_User_004 | 알림 요약          | HOME001-004 | 홈 화면 기능 | 미확인 알림 또는 최근 알림 표시                                                                                     |

# | 요구사항 정의서

| 구분      | 요구사항 ID             | 요구사항명         | 기능 ID             | 기능명    | 세부사항                                                     |
|---------|---------------------|---------------|-------------------|--------|----------------------------------------------------------|
| S/W (앱) | Wallet_User_001     | 사용자 정보 확인     | WALLET001-001     | 지갑 기본  | 사용자 이름과 지갑 잔액을 실시간으로 확인하고 프로필 정보로 본인 계정을 쉽게 식별할 수 있도록 함. |
|         | Wallet_User_002     | 사용자 검색(친구 찾기) | WALLET001-002     | 사용자 검색 | 닉네임, 지갑 주소 등으로 친구를 검색하며 토큰 예치자만 검색 결과에 노출되어 안전성을 확보함.    |
|         | Wallet_User_003     | 송금하기          | WALLET001-003     | 송금     | 003-1→2→3→4→5                                            |
|         | Wallet_User_003-001 | 친구 선택         | WALLET001-003-001 | 송금 단계  | 검색된 목록에서 송금 대상을 선택하고 정보(이름/주소)를 확인해 실수 송금을 방지함.          |
|         | Wallet_User_003-002 | 금액 입력         | WALLET001-003-002 | 송금 단계  | 송금 금액을 입력하며 사용 가능한 잔액과 예상 수수료를 함께 확인할 수 있음.              |
|         | Wallet_User_003-003 | 송금 확인         | WALLET001-003-003 | 송금 단계  | 송금 정보 요약을 팝업으로 보여주고 "송금하시겠습니까?"로 의사를 다시 확인함.             |
|         | Wallet_User_003-004 | 송금 실행         | WALLET001-003-004 | 송금 단계  | 송금 트랜잭션 실행 시 인증 절차를 거치며 성공 시 참조번호를 제공함.                  |
|         | Wallet_User_003-005 | 송금 완료 알림      | WALLET001-003-005 | 송금 단계  | 송금 금액, 일자, 송금자/수신자 정보를 포함한 완료 메시지를 팝업과 알림으로 전달함.         |

## | 요구사항 정의서

| 구분      | 요구사항 ID             | 요구사항명    | 기능 ID             | 기능명   | 세부사항                                               |
|---------|---------------------|----------|-------------------|-------|----------------------------------------------------|
| S/W (앱) | Wallet_User_004     | 요청하기     | WALLET001-004     | 요청    | 004-1→2→3→4                                        |
|         | Wallet_User_004-001 | 친구 선택    | WALLET001-004-001 | 요청 단계 | 요청할 친구를 검색해 선택하고 이름과 지갑 주소를 확인하여 정확한 대상에게 요청함.     |
|         | Wallet_User_004-002 | 금액 입력    | WALLET001-004-002 | 요청 단계 | 받고자 하는 금액을 입력하고 요청 사유나 메모를 함께 입력할 수 있음.            |
|         | Wallet_User_004-003 | 요청 확인    | WALLET001-004-003 | 요청 단계 | 요청 금액과 대상 정보를 최종 확인하고 "요청하시겠습니까?" 팝업으로 의사를 다시 확인함. |
|         | Wallet_User_004-004 | 요청 완료 알림 | WALLET001-004-004 | 요청 단계 | 요청 완료 사실을 상대방에게 알리고 요청 일자, 금액 등 정보를 메시지로 전달함.      |

## | 요구사항 정의서

| 구분      | 요구사항 ID                 | 요구사항명     | 기능 ID                 | 기능명   | 세부사항                                              |
|---------|-------------------------|-----------|-----------------------|-------|---------------------------------------------------|
| S/W (앱) | Wallet_User_005         | 거래 내역 확인  | WALLET001-005         | 거래 확인 | 사용자가 전체 거래 기록을 확인할 수 있으며 친구별 내역 필터링도 가능해 관리가 용이함. |
|         | Wallet_User_005-001     | 친구 선택     | WALLET001-004-001     | 요청 단계 | 특정 친구를 검색해 선택하면 해당 사용자와의 거래 내역만 따로 볼 수 있음.        |
|         | Wallet_User_005-001-001 | 전체 내역 보기  | WALLET001-005-001-001 | 필터 옵션 | 필터 옵션으로 모든 거래 (결제, 입금, 출금)를 한 화면에 종합 조회할 수 있음.    |
|         | Wallet_User_005-001-002 | 결제 내역만 보기 | WALLET001-005-001-002 | 필터 옵션 | 필터를 통해 결제 카테고리 내역만 따로 모아 확인할 수 있음.                |
|         | Wallet_User_005-001-003 | 입금 내역만 보기 | WALLET001-005-001-003 | 필터 옵션 | 필터 옵션으로 입금 내역만 선택해 상세 내역을 따로 볼 수 있음.              |
|         | Wallet_User_005-001-004 | 출금 내역만 보기 | WALLET001-005-001-004 | 필터 옵션 | 필터를 적용해 출금 내역만 모아 확인하고 관리할 수 있음.                  |



# | 요구사항 정의서

| 구분      | 요구사항 ID             | 요구사항명     | 기능 ID             | 기능명    | 세부사항                                                       |
|---------|---------------------|-----------|-------------------|--------|------------------------------------------------------------|
| S/W (앱) | Budget_User_001     | 월별 예산 설정  | BUDGET001-001     | 예산 설정  | 사용자가 매월 원하는 지출 한도를 직접 입력하고 필요 시 수정하여 지출을 관리할 수 있음.         |
|         | Budget_User_001-001 | 월 예산 입력   | BUDGET001-001-001 | 한도 입력  | 월별로 설정할 한도 금액을 입력하는 필드를 제공하여 사용자 맞춤 예산 관리가 가능함.            |
|         | Budget_User_001-002 | 예산 수정     | BUDGET001-001-002 | 한도 수정  | 이미 설정된 한도를 상황에 따라 변경할 수 있어 계획적인 소비 조정이 가능함.                |
|         | Budget_User_002     | 예산 사용률 확인 | BUDGET001-002     | 예산 사용률 | 현재 사용 금액과 잔여 금액을 그래프 등으로 시각화해 한눈에 사용률을 확인할 수 있음.           |
|         | Budget_User_003     | 소비 리포트    | BUDGET001-003     | 리포트    | 카테고리별 사용 비율(파이차트) + 상위 3개 항목 + 기간 선택(월/주)                  |
|         | Budget_User_003-001 | 기간 선택     | BUDGET001-003-001 | 기간 필터  | 월간/주간 리포트로 기간을 선택해 원하는 기간의 소비 패턴만 따로 분석할 수 있음.             |
|         | Budget_User_003-002 | 카테고리별 요약  | BUDGET001-003-002 | 요약 데이터 | 카테고리별 지출 데이터를 한눈에 파악할 수 있도록 파이차트로 요약 제공.                   |
|         | Budget_User_003-003 | 요약 코멘트    | BUDGET001-003-003 | 한 줄 알림 | “이번 달 가장 많이 사용한 카테고리는 XXX입니다.”와 같은 한 줄 알림으로 소비 습관을 인지시켜 줌. |

# | 요구사항 정의서

| 구분      | 요구사항 ID              | 요구사항명      | 기능 ID               | 기능명    | 세부사항                                          |
|---------|----------------------|------------|---------------------|--------|-----------------------------------------------|
| S/W (앱) | Setting_User_001     | 사용자 정보 관리  | SETTINGS001-001     | 사용자 정보 | 사용자가 앱 내에서 기본 정보를 직접 확인하고 필요할 때 수정할 수 있음.     |
|         | Setting_User_001-001 | 이름(닉네임) 수정 | SETTINGS001-001-001 | 이름 수정  | 사용자 이름 또는 닉네임을 입력 필드에서 변경해 최신 정보로 유지 가능.      |
|         | Setting_User_001-002 | 이메일 확인     | SETTINGS001-001-002 | 이메일    | 등록된 이메일 주소를 보고 필요하다면 수정하여 연락처 정보를 최신 상태로 관리함. |
|         | Setting_User_001-003 | 지갑 주소 확인   | SETTINGS001-001-003 | 지갑 주소  | 내 지갑 주소를 확인하고 복사해 다른 사용자와 공유할 수 있음.           |
|         | Settings_User_002    | 이용 약관 보기   | SETTINGS001-002     | 약관     | 서비스 이용약관과 개인정보 처리방침을 언제든지 앱에서 확인 가능.          |
|         | Settings_User_003    | 개발진 정보 보기  | SETTINGS001-003     | 개발진    | 앱을 만든 개발팀의 간략한 소개와 연락 정보 등을 볼 수 있음.           |

# | 요구사항 정의서

| 구분      | 요구사항 ID               | 요구사항명    | 기능 ID               | 기능명   | 세부사항                                            |
|---------|-----------------------|----------|---------------------|-------|-------------------------------------------------|
| S/W (앱) | Settings_User_004     | 푸시 알림 설정 | SETTINGS001-004     | 알림 설정 | 앱에서 푸시 알림 수신 여부를 ON/OFF로 설정하고 알림 종류별로 선택할 수 있음. |
|         | Settings_User_005     | 공지사항 확인  | SETTINGS001-005     | 공지사항  | 서비스 공지 목록을 최신순으로 확인하고 필요한 공지사항 세부 내용을 열람할 수 있음. |
|         | Settings_User_006     | 1:1 문의하기 | SETTINGS001-006     | 문의    | 사용자가 개인 문의를 작성하고 답변 진행 상황을 앱에서 바로 확인할 수 있음.     |
|         | Settings_User_007     | 로그아웃     | SETTINGS001-007     | 로그아웃  | 세션을 안전하게 종료하고 로그인 상태를 해제하여 개인정보를 보호함.           |
|         | Settings_User_008     | 회원탈퇴     | SETTINGS001-008     | 탈퇴    | 계정을 완전히 삭제하는 탈퇴 절차를 제공하며 관련 동의 팝업으로 의사를 재확인함.   |
|         | Settings_User_008-001 | 탈퇴 동의 팝업 | SETTINGS001-008-001 | 탈퇴 팝업 | 회원탈퇴 시 yes/no 버튼이 있는 팝업으로 계정 삭제 의사를 다시 확인함.     |

# | 요구사항 정의서

| 구분      | 요구사항 ID           | 요구사항명    | 기능 ID            | 기능명       | 세부사항                                                       |
|---------|-------------------|----------|------------------|-----------|------------------------------------------------------------|
| S/W (앱) | Moto_User_001     | 경로 입력    | MOTOR001-001     | 경로 입력     | 출발지와 도착지를 검색해 원하는 이동 경로를 설정하고 지도에 표시할 수 있음.                |
|         | Moto_User_001-001 | 출발지 입력   | MOTOR001-001-001 | 출발지 장소 입력 | API를 통해 출발지 장소를 검색하고 선택할 수 있음.                             |
|         | Moto_User_001-002 | 도착지 입력   | MOTOR001-001-002 | 도착지 장소 입력 | 도착지 장소도 API로 불러와 입력 후 경로에 반영함.                             |
|         | Moto_User_001-003 | 선택 경로 표시 | MOTOR001-001-003 | 경로 표시     | 선택된 출발지와 도착지를 기반으로 지도를 통해 경로를 시각적으로 표시함.                   |
|         | Moto_User_002     | 차량 호출    | MOTOR001-002     | 호출 기능     | 설정된 경로를 기준으로 호출을 실행하고 호출 가능한 차량 목록과 현재 상태를 실시간으로 확인할 수 있음. |

# | 요구사항 정의서

| 구분      | 요구사항 ID                   | 요구사항명    | 기능 ID                    | 기능명    | 세부사항                                          |
|---------|---------------------------|----------|--------------------------|--------|-----------------------------------------------|
| S/W (앱) | Moto_User_003             | 결제 QR 스캔 | MOTOR001-003             | 결제 인증  | 이용 종료 시 QR 코드를 스캔해 결제를 인증하고 자동으로 결제가 실행됨.     |
|         | Moto_User_003-001         | 스캔 실행    | MOTOR001-003-001         | 스캐너 실행 | QR 스캐너를 실행해 사용자가 도착지에서 결제용 QR코드를 스캔할 수 있음.    |
|         | Moto_User_003-001-001     | 결제 승인 처리 | MOTOR001-003-001-001     | 결제 승인  | QR 인증이 성공하면 결제 트리거가 실행되어 결제 프로세스가 시작됨.        |
|         | Moto_User_003-001-001-001 | 자동 결제    | MOTOR001-003-001-001-001 | 결제 처리  | QR 인증 후 실제 이동 거리를 계산해 사용자의 토큰이나 요금이 자동으로 차감됨. |
|         | Moto_User_003-001-001-002 | 결제 알림    | MOTOR001-003-001-001-002 | 결제 확인  | 결제 성공 시 트랜잭션 ID 등 결제 결과를 알림으로 사용자에게 안내함.      |
|         | Moto_User_003-001-001-003 | 결제 실패 대응 | MOTOR001-003-001-001-003 | 예외 처리  | QR 인증 실패 또는 결제 오류 발생 시 재시도 흐름과 이용 제한 방안을 제공함. |

# | 요구사항 정의서

| 구분      | 요구사항 ID        | 요구사항명              | 기능 ID       | 기능명                 | 세부사항                                                                     |
|---------|----------------|--------------------|-------------|---------------------|--------------------------------------------------------------------------|
| S/W (웹) | Auth_Admin_001 | 관리자 로그인            | AUTH001-001 | 관리자 계정 로그인          | 관리자 ID / 비밀번호 입력해 시스템에 로그인                                               |
|         | Dash_Admin_001 | 대시보드 조회            | DASH001-001 | 전체 거래 내용 요약 대시보드 조회 | 결제건수, 이체건수, 거래량, 월별 거래량 등 시각적인 차트나 그래프, 숫자로 표현                           |
|         | Sup_Admin_001  | 발행량 조회             | SUP001-001  | 토큰 발행량 조회           | 현재 총 발행된 토큰 수량 조회                                                        |
|         | Txn_Admin_001  | 거래내역 전체 조회 및 거래 취소 | TXN001-001  | 사용자 거래 내역 전체 조회     | 모든 사용자의 거래내역을 목록으로 제공<br>각 거래의 상태(예: 정상, 실패), 처리 완료 여부, 취소 가능 여부를 명확히 표시 |
|         | Txn_Admin_001  | 거래내역 전체 조회 및 거래 취소 | TXN001-002  | 해당 사용자 거래 취소        | 조회된 거래 내역 각 항목의 오른쪽에 '거래 취소' 버튼을 제공<br>이 버튼을 클릭하여 해당 거래를 취소 처리           |

## | 요구사항 정의서

| 구분      | 요구사항 ID        | 요구사항명           | 기능 ID       | 기능명                     | 세부사항                                                                               |
|---------|----------------|-----------------|-------------|-------------------------|------------------------------------------------------------------------------------|
| S/W (웹) | Noti_Admin_001 | 사용자 앱 알림        | NOTI001-001 | 알림 내용 입력                | 관리자가 사용자에게 보낼 푸시 알림 메시지 입력<br>메시지 내용 입력시 알림버튼 활성화                                  |
|         | Noti_Admin_001 | 사용자 앱 알림        | NOTI001-002 | 앱 푸시알림 버튼               | 알림 메시지가 적혀있을 때<br>알림 버튼 활성화 및 클릭시 알림 전송                                            |
|         | Noti_Admin_002 | 사용자 앱 공지        | NOTI002-001 | 공지 내용 입력                | 관리자가 사용자에게 등록할 공지 메시지 입력<br>메시지 내용 입력시 공지버튼 활성화                                    |
|         | Noti_Admin_002 | 사용자 앱 공지        | NOTI002-002 | 앱 공지 버튼                 | 공지 메시지가 적혀 있을 때<br>공지 버튼 활성화 및 클릭시 공지 전송                                           |
|         | Acc_Admin_001  | 사용자 조회          | ACC001-001  | 앱 사용자 조회                | 사용자의 이름과 생년월일을 기준으로<br>특정 사용자를 검색하고 상세 정보를 조회                                      |
|         | Acc_Admin_002  | 사용자 지갑 조회       | ACC002-001  | 사용자별 지갑잔액 조회            | 특정 사용자를 선택하면 해당 사용자의<br>지갑 잔액을 즉시 확인<br>잔액은 원(KRW) 또는 토큰 수량과 같은<br>단위와 함께 표시       |
|         | Acc_Admin_002  | 사용자 지갑 조회       | ACC002-002  | 사용자 지갑 토큰 지급            | 관리자가 지급할 토큰의 수량을 직접 입력하여<br>특정 사용자에게 토큰 지급                                         |
|         | Acc_Admin_002  | 사용자 지갑 조회       | ACC002-003  | 사용자 지갑 특정 거래 회수         | 사용자의 거래 내역에서 특정 항목을 클릭하여<br>지급되었던 토큰이나 금액을 회수 처리                                   |
|         | Acc_Admin_003  | 전체 사용자 거래 내역 조회 | ACC003-001  | 전체 사용자 지갑 정보 및 거래 내역 조회 | 전체 사용자의 지갑 정보와 모든 거래 내역을 한 번에 조회, 조회 항목에는 사용자 정보, 거래 일시, 거래 금액, 거래 후 잔액, 거래 상태를 포함 |

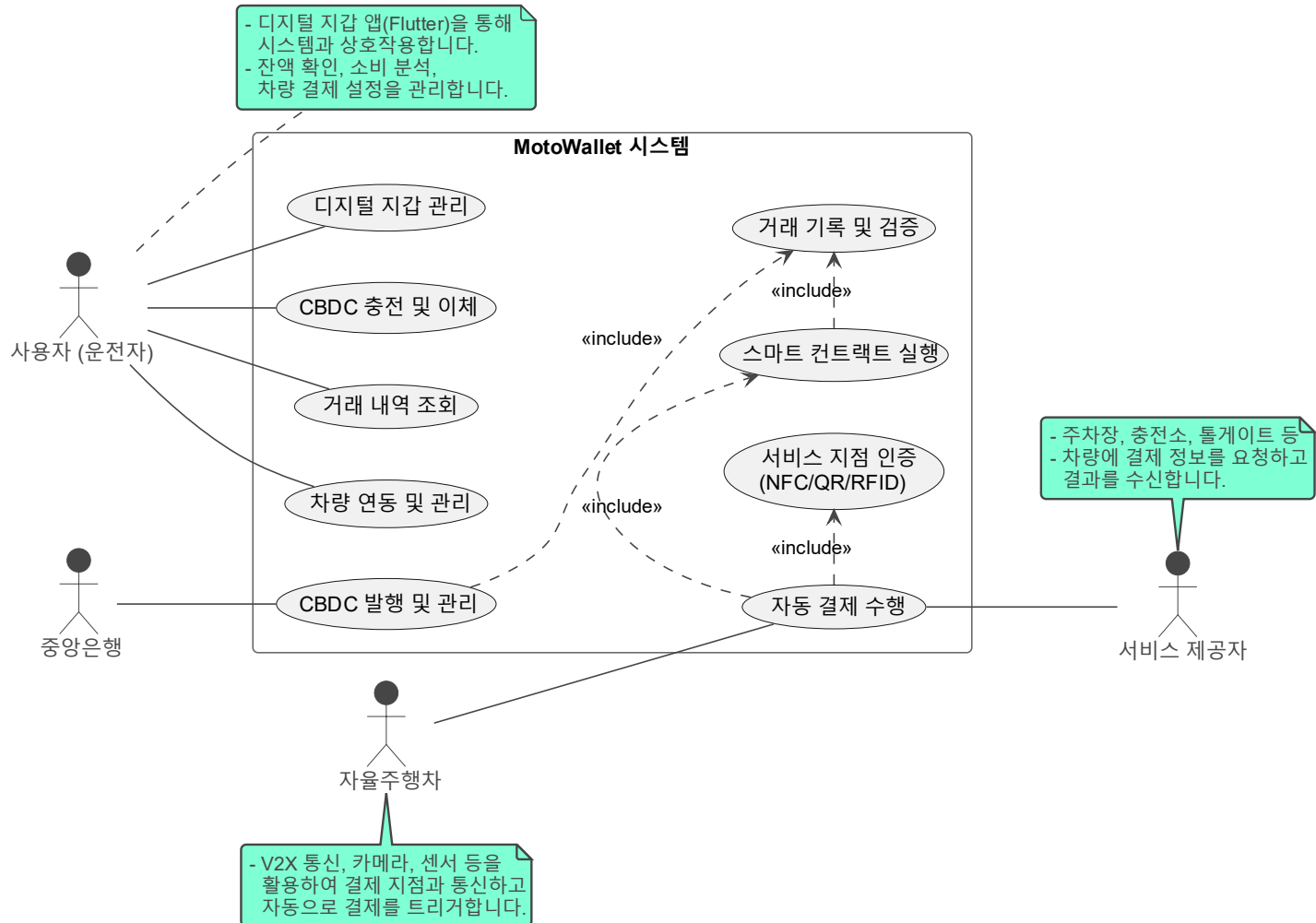
# | 요구사항 정의서

| 구분  | 요구사항 ID      | 요구사항명       | 기능 ID       | 기능명         | 세부사항                                                        |
|-----|--------------|-------------|-------------|-------------|-------------------------------------------------------------|
| H/W | Moto_Car_001 | 차량 주행 제어    | Auto001-001 | 자율 주행 제어    | 차량 카메라 영상을 OpenCV로 분석하여 경로를 판단하고, 서보모터와 DC모터를 제어하여 자율주행을 수행 |
|     | Moto_Car_001 | 차량 주행 제어    | Auto001-002 | 조향 및 속도 제어  | 좌우 방향 조절은 서보모터로, 전진·후진 및 속도 제어는 DC모터를 통해 수행                 |
|     | Moto_Car_002 | 사용자 상태 표시   | Auto002-001 | 상태 디스플레이 표시 | 차량의 상태(예약중/탑승중/도착 등)를 LED 색상 또는 LCD 문구로 표시                  |
|     | Moto_Car_003 | 승/하차 제어     | Auto003-001 | 마커 인식 정지    | 카메라가 인식한 승하차 마커(예: 빨강, 초록 등)를 기준으로 차량이 정지 위치를 판단            |
|     | Moto_Car_003 | 승/하차 제어     | Auto003-002 | 라이다 거리 보정   | 라이다 센서를 통해 정지 위치를 보정하여 정확한 위치에 정차 가능                        |
|     | Moto_Car_004 | 사용자 인증 및 결제 | Auto004-001 | QR코드 인증 처리  | 차량 외부 QR코드를 사용자가 앱으로 스캔하면 차량과 앱 간의 인증 가능                    |
|     | Moto_Car_004 | 사용자 인증 및 결제 | Auto004-002 | 결제 트리거 연동   | QR 인증 완료 후 탑승 상태가 확인되면 결제 트리거가 활성화되고 블록체인 연동 서버로 전달         |
|     | Moto_Car_005 | 앱 통신 연동     | Auto005-001 | 앱-차량 상태 동기화 | Wi-Fi를 통해 앱과 차량 간의 호출, 인증, 주행 상태를 실시간 동기화                   |

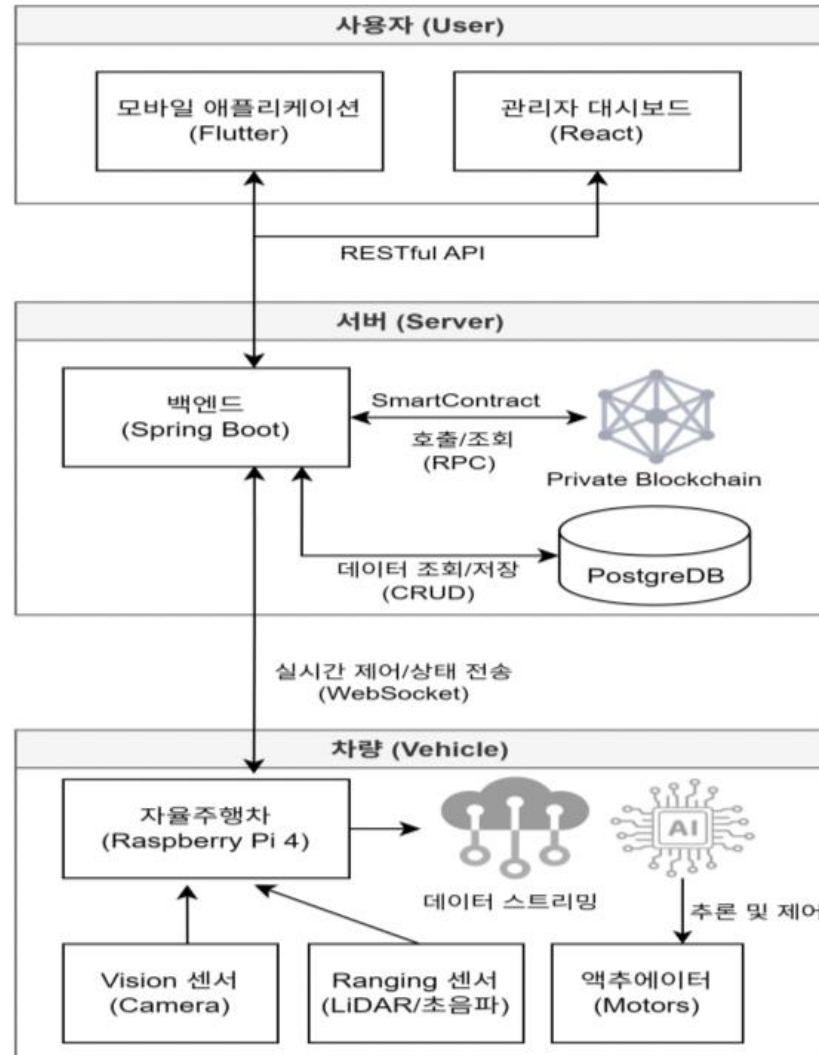


# | 유즈케이스 정의서

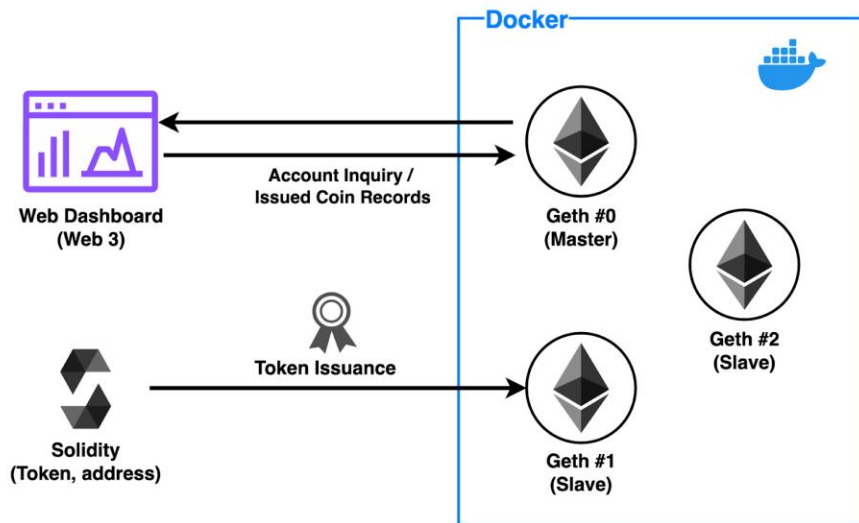
## MotoWallet: CBDC 기반 자율주행차 결제 시스템 유즈케이스



## | 시스템 설계도

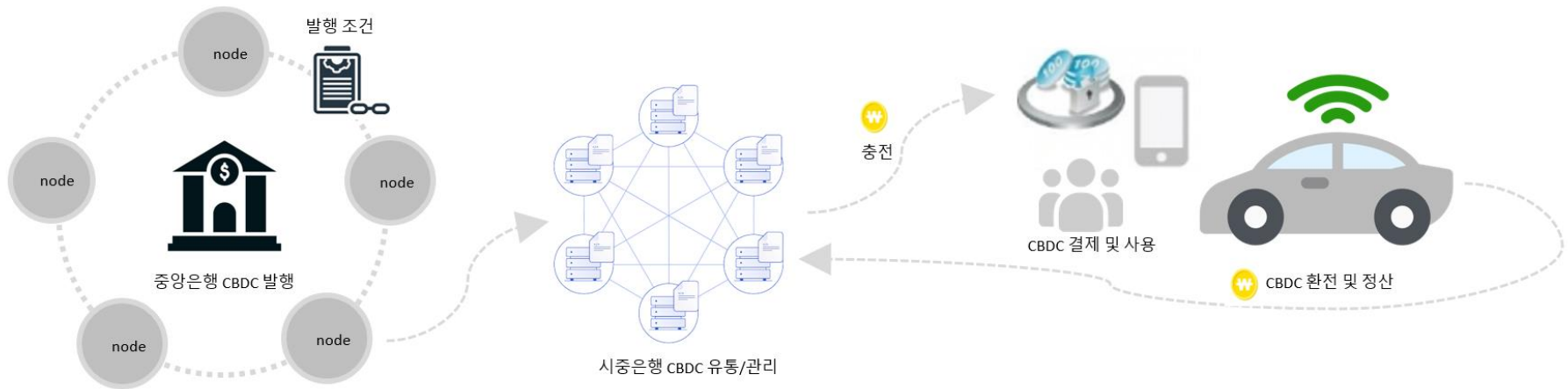


## | 서비스 구성도 - 서비스 시나리오



|                          |                                                                                                                                                                                                                                                                                                                                                                                                                         |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>블록체인 종류<br/>(eth)</b> | Platform: Ethereum Private Network<br>Client: Geth v1.12.0<br>Consensus: Clique (Proof-of-Authority)                                                                                                                                                                                                                                                                                                                    |
| <b>인프라 환경</b>            | Docker 기반 멀티 컨테이너 구성                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Token 정보</b>          | Token Name: MotoToken<br>Token Symbol: MYT<br>총 발행량 (Total Supply): 1,000,000 MYT<br>Decimals: 18<br><br>총 발행량 (wei) = $1,000,000 \times 10^{18} = 1000000000000000000000000$                                                                                                                                                                                                                                           |
| <b>서비스<br/>시나리오</b>      | <ol style="list-style-type: none"> <li>1) 사용자가 Geth 노드에 새 지갑을 만들어 계정을 생성한다.</li> <li>2) 관리자가 Geth 노드에서 스마트컨트랙트를 배포해 토큰을 발행한다.</li> <li>3) 사용자가 토큰을 받으려면 Geth에 트랜잭션을 보내 토큰 전송을 요청한다.</li> <li>4) Geth가 트랜잭션을 블록에 포함해 토큰 잔액을 업데이트한다.</li> <li>5) 사용자가 잔액 조회를 요청하면 Geth가 스마트컨트랙트의 상태값을 반환한다.</li> <li>6) 사용자가 송금이나 결제를 하면 Geth가 해당 트랜잭션을 처리해 블록에 기록한다.</li> <li>7) 대시보드는 Geth에서 블록, 거래 내역, 계좌 상태를 받아와 화면에 보여준다.</li> </ol> |

## | 서비스 흐름도



- ✓ CBDC 기반 모빌리티 토큰 서비스 설명 본 프로젝트는 실제 중앙은행 및 정부의 인가를 받은 사업이 아니며, 시연 목적의 가정임. 중앙은행이 최초 사업자 인가 및 셋업 인가를 승인했다고 가정하고, 1,000,000개의 초기 공급량 토큰으로 서비스를 시작함.
- ✓ 발행된 토큰은 시중은행을 통해 유통·관리되며, 사용자는 모바일 지갑을 통해 충전·결제·송금 기능을 이용할 수 있음.
- ✓ 모빌리티 서비스에서는 이동량에 따라 토큰이 자동 정산되며, 모든 내역은 블록체인에 기록되어 **투명성을 보장함**.
- ✓ 향후 실제 런칭 시에는 중앙은행 및 관련 기관과 협의하여 인가·승인을 획득하고, 확장된 금융·모빌리티 서비스를 제공할 수 있을 것으로 기대함.

## | 하드웨어 구성도

출력장치



DC 모터



LED



BUZZER

기반 플랫폼



Raspberry pi 4B

활용 플랫폼



추론 모델



WebSocket

앱, 웹 통신

입력장치

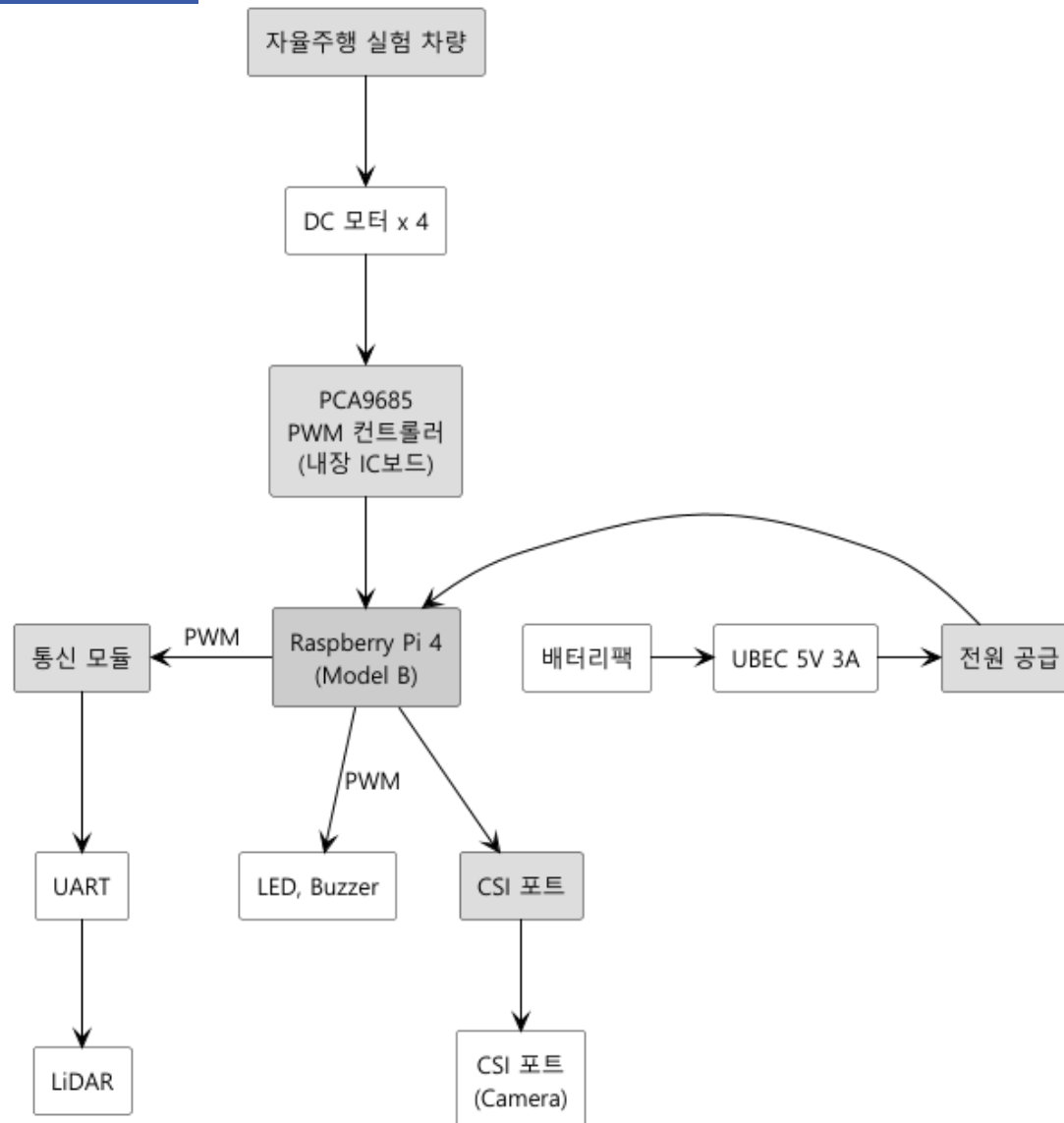


pi camera

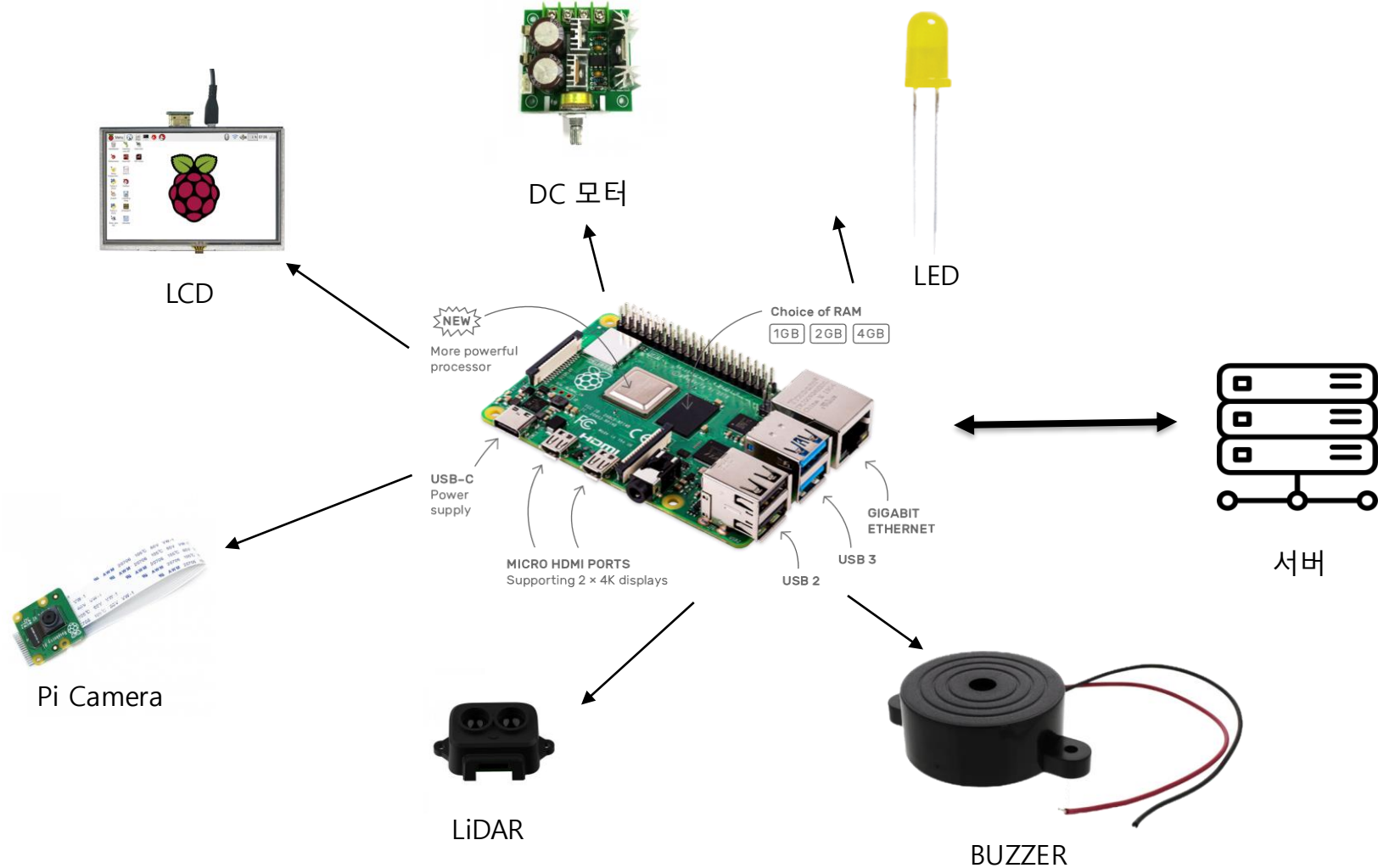


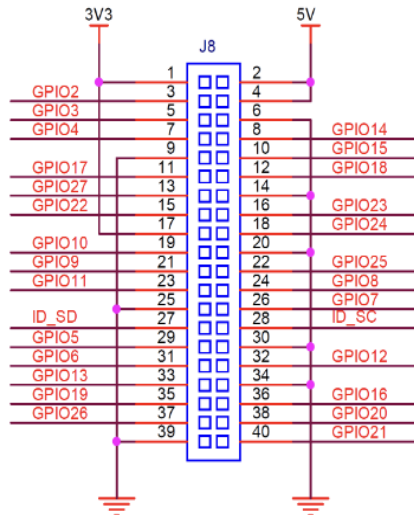
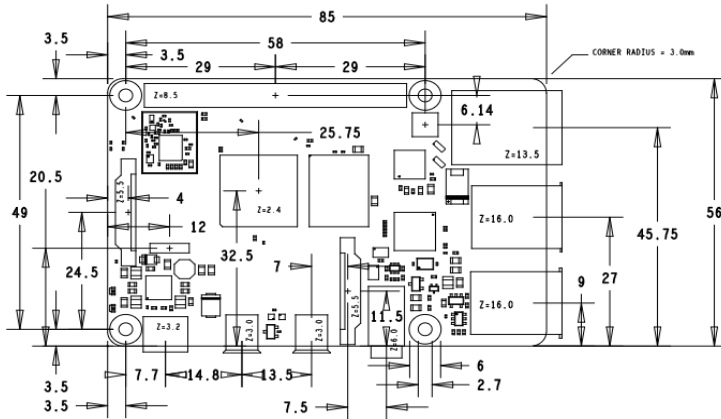
Lidar sensor

## | 하드웨어 설계도



# | 하드웨어/센서 구성도





| 구성 요소               | 연결 방식 / 핀        | 설명                             |
|---------------------|------------------|--------------------------------|
| LED 1               | GPIO 16          | 차량 상태 표시용 LED (주행 /알림용)        |
| LED 2               | GPIO 26          | 차량 상태 표시용 LED (LED1과 동일)       |
| 모터 좌측 PWM (L_Motor) | GPIO 18 (PWM)    | 좌측 모터 속도 제어                    |
| 모터 좌측 AIN1          | GPIO 22          | 좌측 모터 회전 방향 제어 (H-Bridge)      |
| 모터 좌측 AIN2          | GPIO 27          | 좌측 모터 회전 방향 제어 (H-Bridge)      |
| 모터 우측 PWM (R_Motor) | GPIO 23 (PWM)    | 우측 모터 속도 제어                    |
| 모터 우측 BIN1          | GPIO 25          | 우측 모터 회전 방향 제어 (H-Bridge)      |
| 모터 우측 BIN2          | GPIO 24          | 우측 모터 회전 방향 제어 (H-Bridge)      |
| 부저 (Buzzer)         | GPIO 20          | 경고/알람용 부저                      |
| 카메라                 | USB / CSI        | Picamera2로 영상 캡처 (GPIO 미사용)    |
| 라이다 (LiDAR)         | UART / I2C / USB | 거리 측정 센서. 장애물 감지 및 SLAM/주행 보조용 |



## | 정보 구조도 (사용자 모바일 앱)



## | 정보 구조도 (관리자 웹)



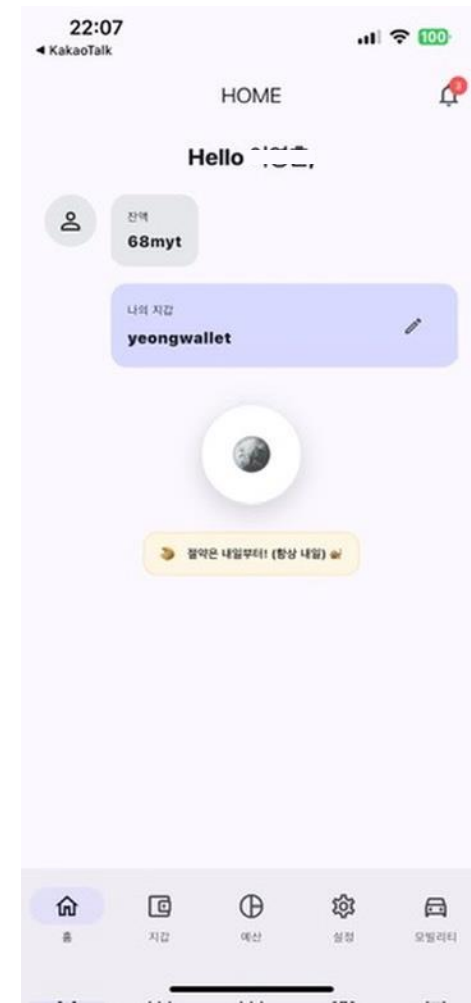
## | 화면 설계서 - 사용자 인터페이스(sw)



▲ 카카오 로그인

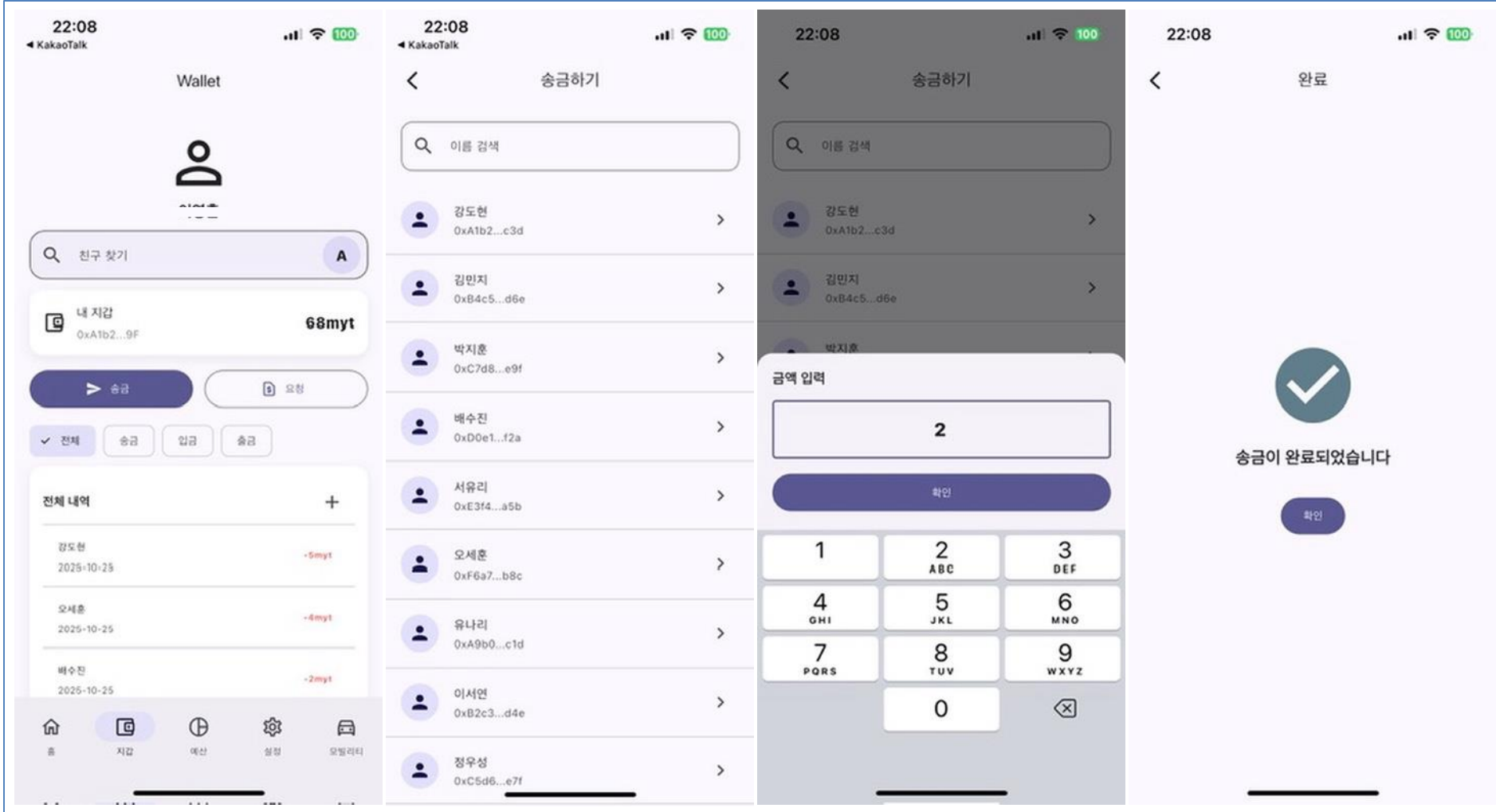


▲ 백엔드 연동 후  
토큰 및 지갑 발행



▲ 홈 화면  
카카오 로그인을 통해  
사용자 이름 받아오기

# | 화면 설계서 - 사용자 인터페이스(sw)



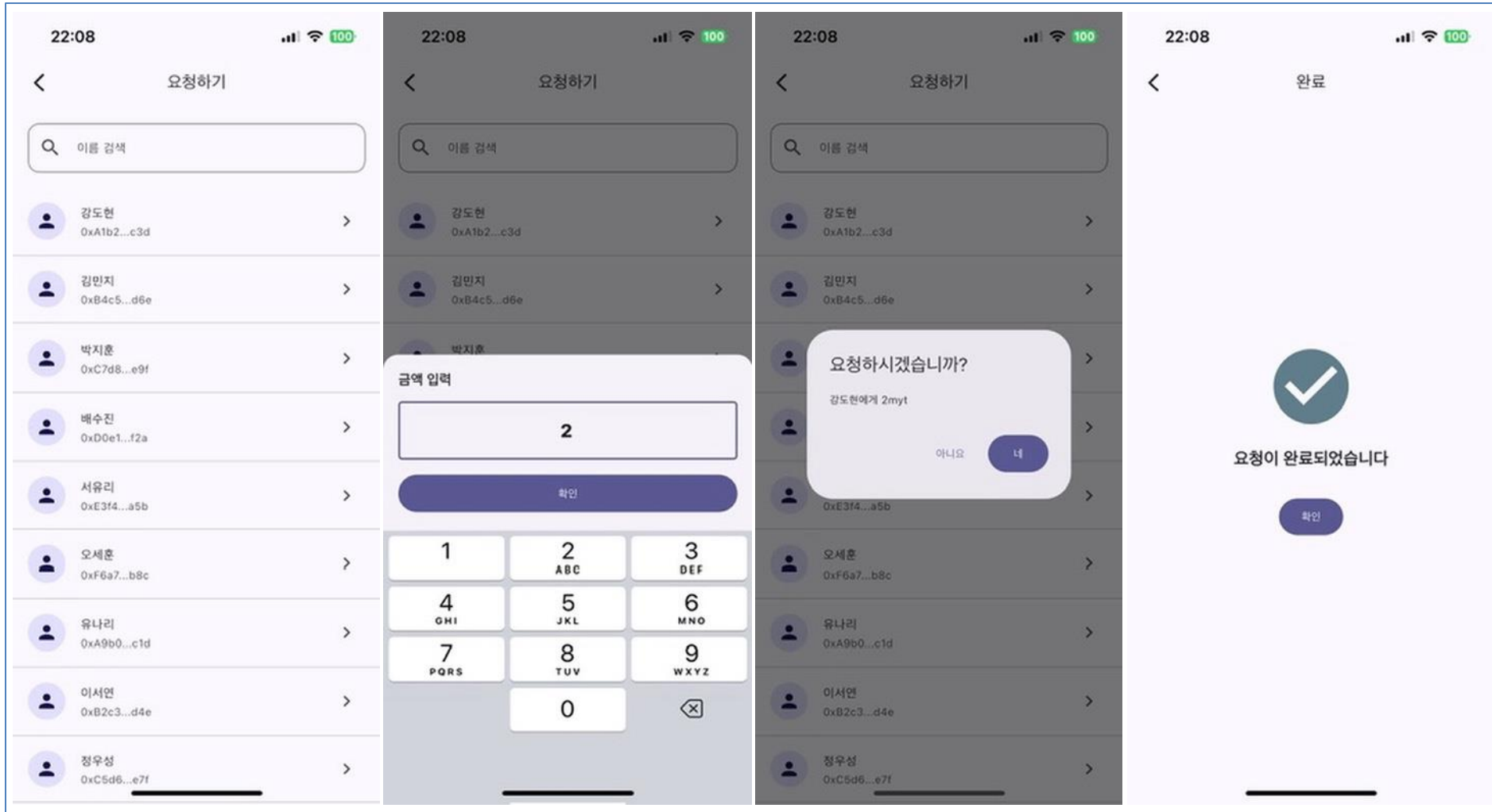
▲ 사용자 화면

▲ 다른 사용자에게 송금하기

▲ 금액 입력

▲ 송금 완료 표시

## | 화면 설계서 - 사용자 인터페이스(UI)



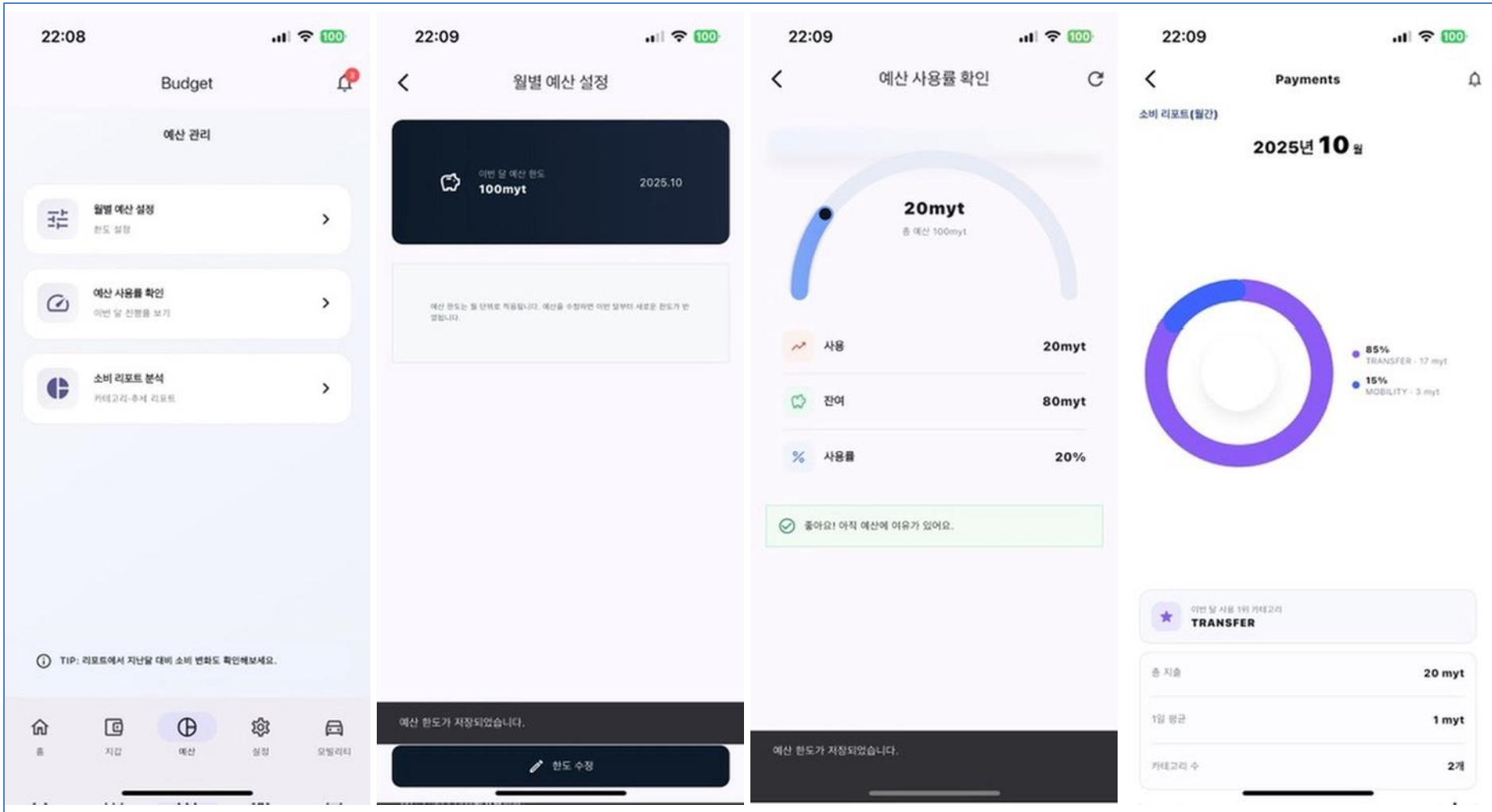
▲ 송금 요청하기

▲ 다른 사용자에게 금액 요청하기

▲ 요청 확인 페이지

▲ 요청 완료 표시

# | 화면 설계서 - 사용자 인터페이스(sw)



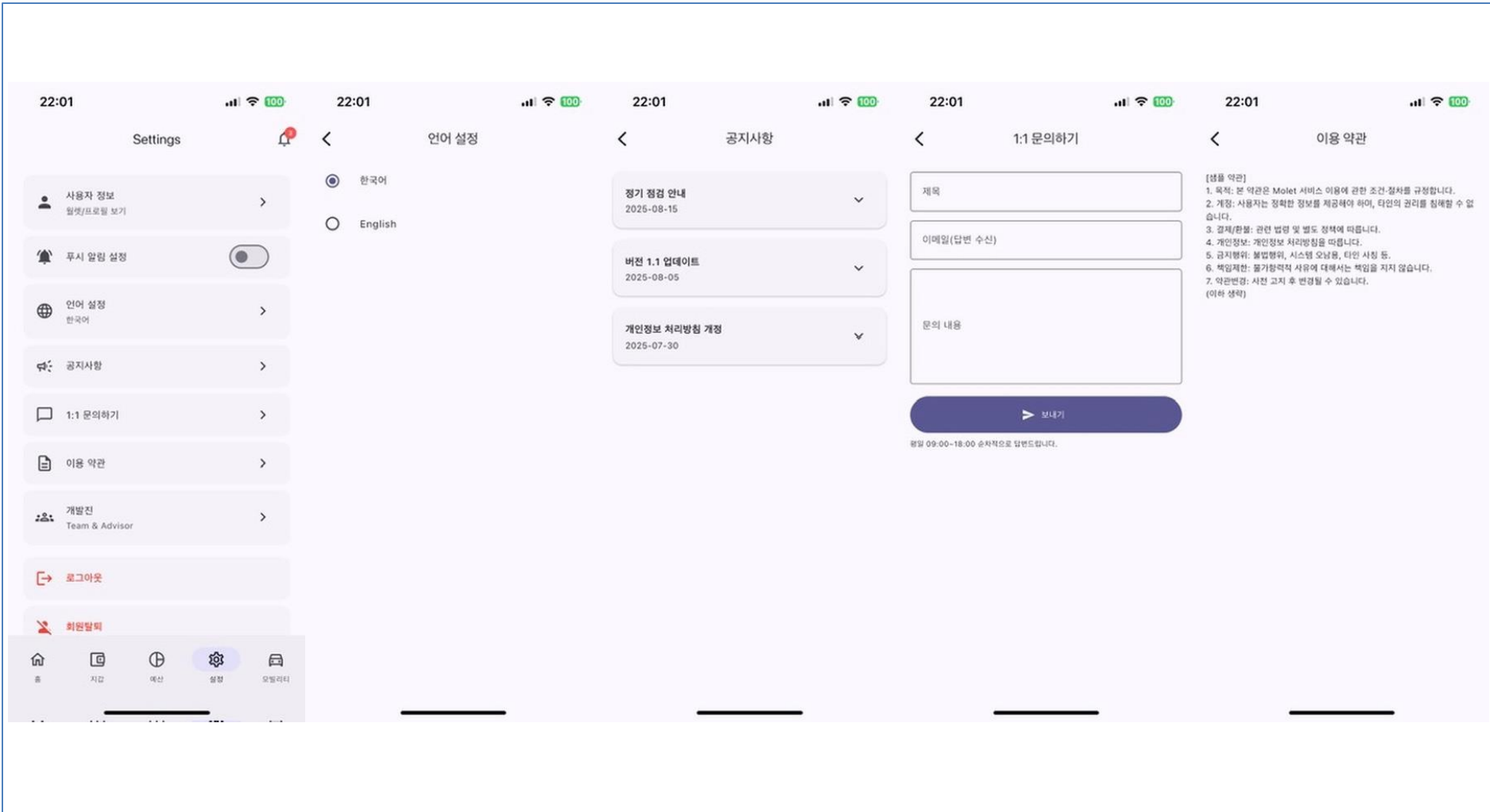
▲ 예산 관리

▲ 월별 예산 설정

▲ 예산 사용률 확인

▲ 월별 소비 리포트

# | 화면 설계서 - 사용자 인터페이스(SW)



▲ 환경 설정

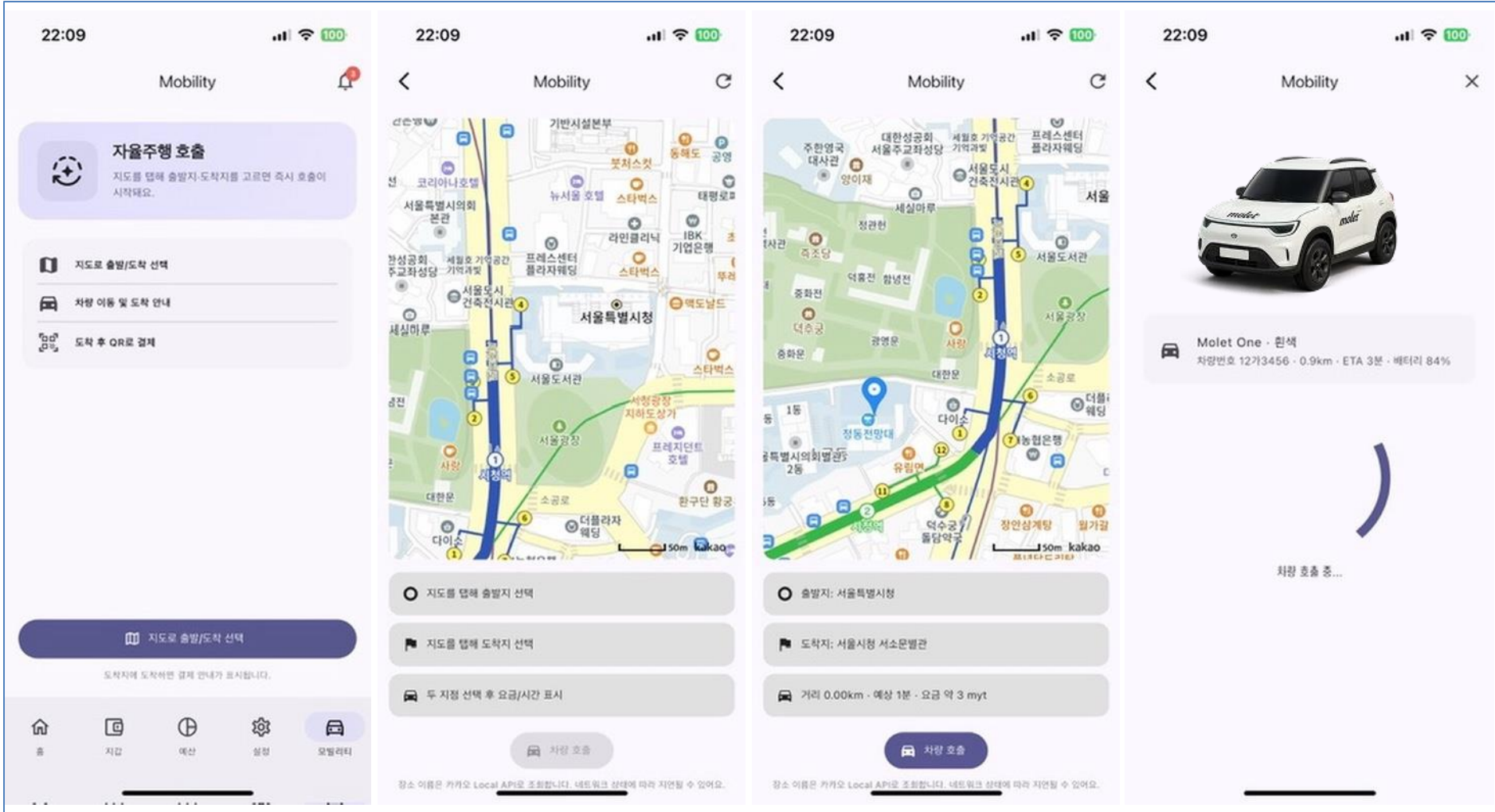
▲ 언어 설정

▲ 공지사항

▲ 문의하기

▲ 이용 약관

# | 화면 설계서 - 사용자 인터페이스(sw)



▲ 자율주행 호출

▲ 지도에 핀을 이용하여  
출발지 선택

▲ 도착지 선택  
거리별 차등 금액 지정

▲ 가까운 자율 주행 차량  
호출



## | 화면 설계서 - 사용자 인터페이스(UI)



▲ 탑승 QR 스캔

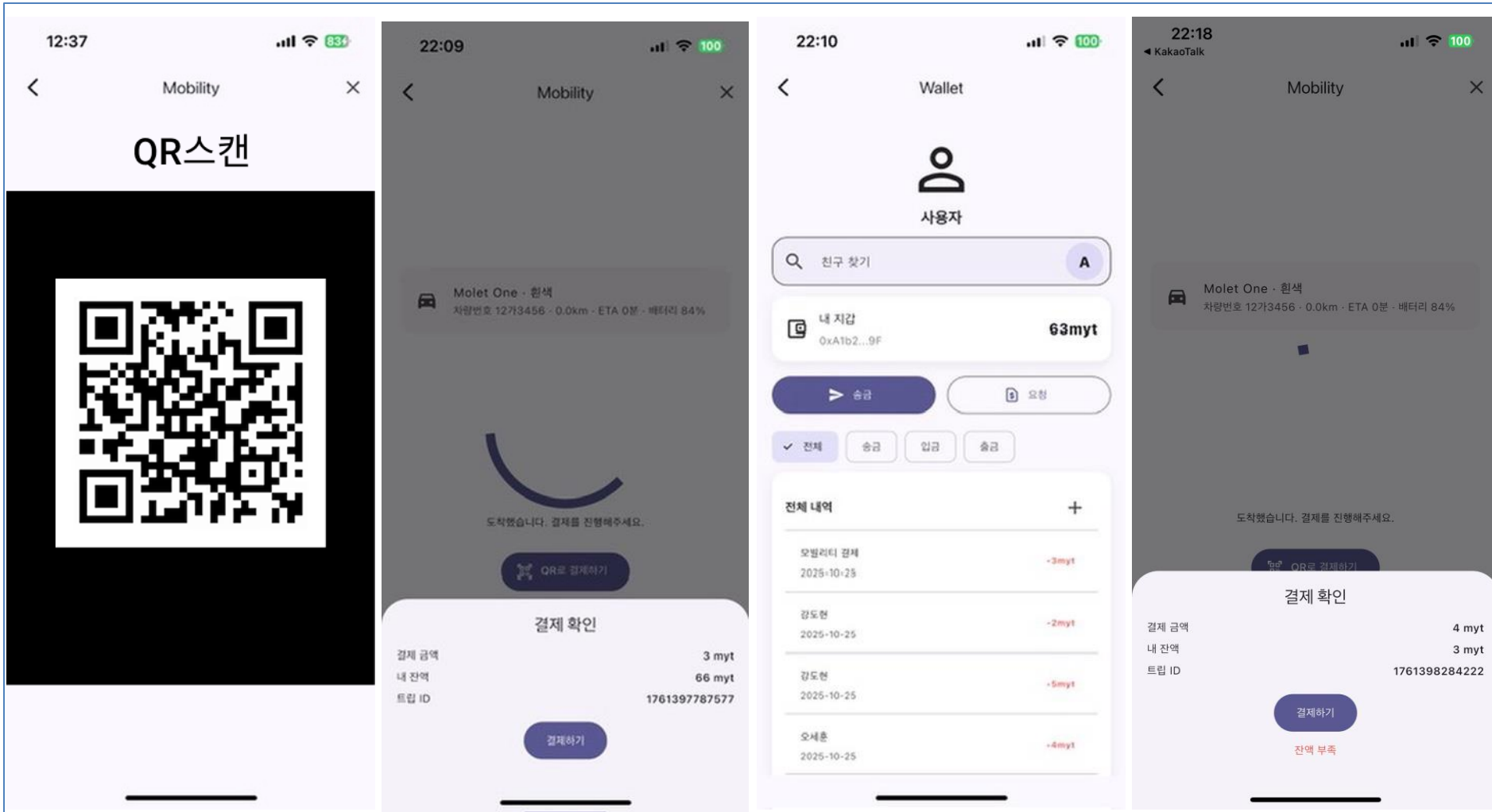


▲ 이동 거리 계산



▲ 도착 후 결제 페이지

# | 화면 설계서 - 사용자 인터페이스(sw)



▲ 도착 및 결제  
QR 스캔

▲ 결제 확인 페이지

▲ 결제 후  
사용자 페이지

▲ 결제 시  
잔액 부족 페이지

## | 화면 설계서 - 관리자 인터페이스(sw)

### MOTOWALLET

Login

Email

Password

Enter password

Sign up

Log in

### HELLO! MOTOWALLET

Name

Enter Name

Email

Enter new Email

Password

Enter password

Password check

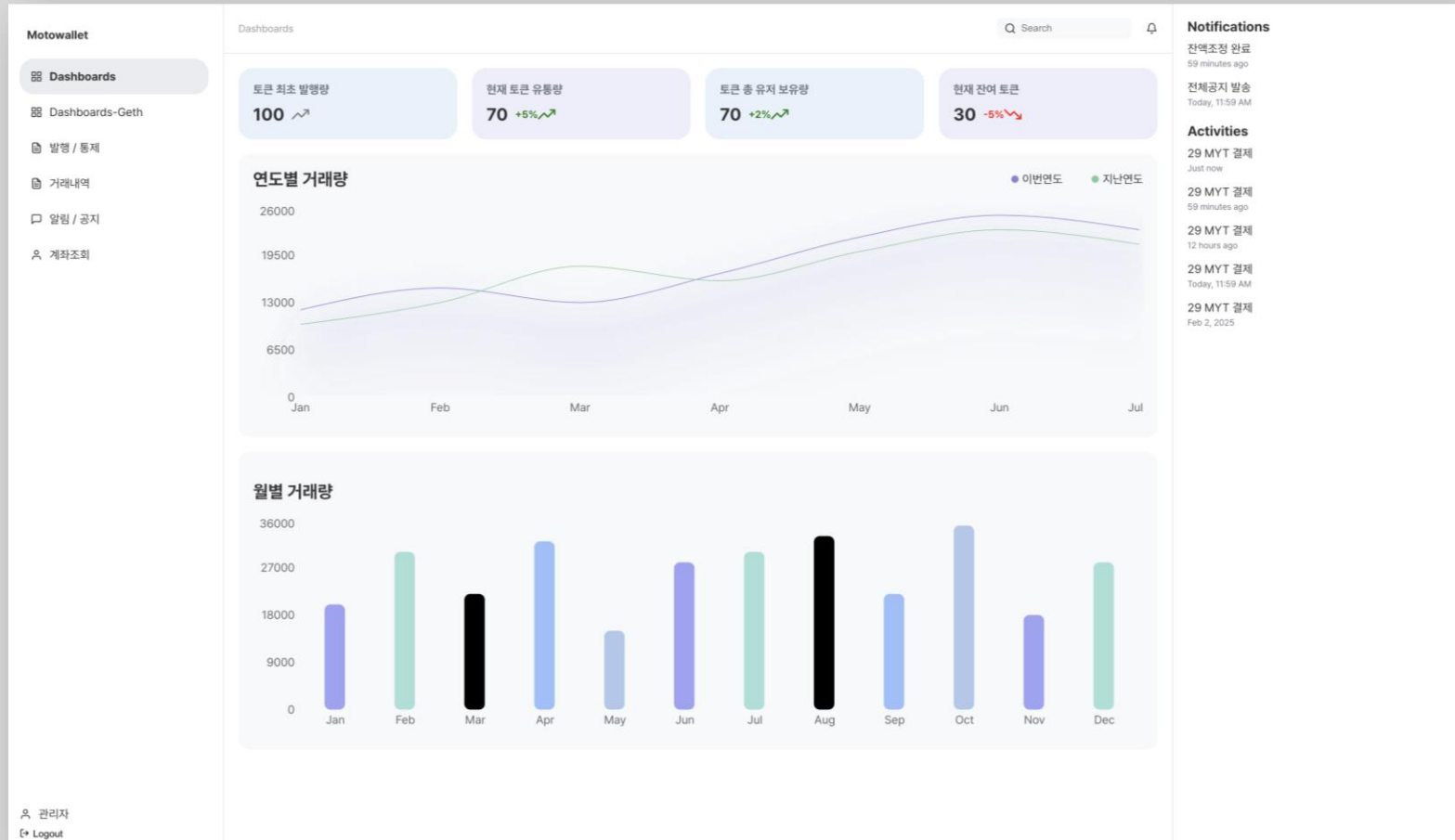
Enter password

Sign up

▲ 관리자 계정(웹) 로그인

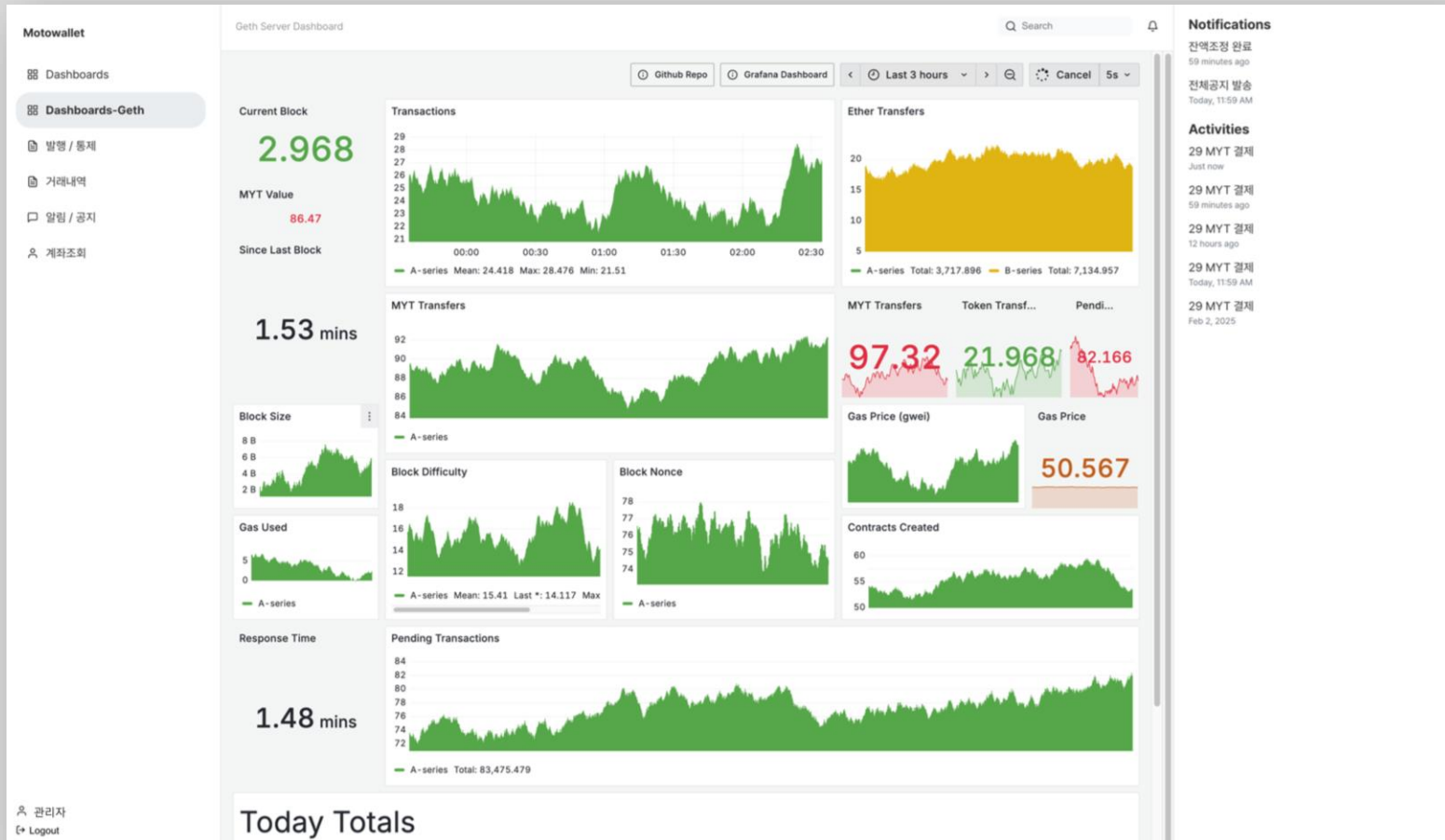
▲ 관리자 계정(웹) 이메일 회원가입

# | 화면 설계서 - 관리자 인터페이스(sw)



▲ 관리자 사용자 거래 관리 대시보드

# | 화면 설계서 - 관리자 인터페이스(sw)



▲ 관리자 geth 대시보드

# | 화면 설계서 - 관리자 인터페이스(sw)

Motowallet

Dashboards

Dashboards-Geth

발행 / 통제

거래내역

알림 / 공지

계좌조회

Issuance / Control

발행 | 통제

MotoToken CBDC

현재 발행량은 100입니다.

발행 및 통제를 위해선 아래 공간을 참고해 주세요.

발행량 또는 통제량을 입력하세요.

단위

발행

통제

가맹점 목록

| 상호명      | 대표자 | 승인일        | 상태 | 가맹취소 |
|----------|-----|------------|----|------|
| 택시(4885) | 최유진 | 2025.06.16 | 정상 | 취소하기 |
| 택시(4885) | 이서연 | 2025.06.16 | 정상 | 취소하기 |

가맹점 등록

가맹점 등록을 위해선 아래 내용을 입력해 주세요.

상호명:

대표자:

초기화

등록

Search

Notifications

잔액조정 완료

59 minutes ago

전체공지 발송

Today, 11:59 AM

Activities

29 MYT 결제

Just now

29 MYT 결제

59 minutes ago

29 MYT 결제

12 hours ago

29 MYT 결제

Today, 11:59 AM

29 MYT 결제

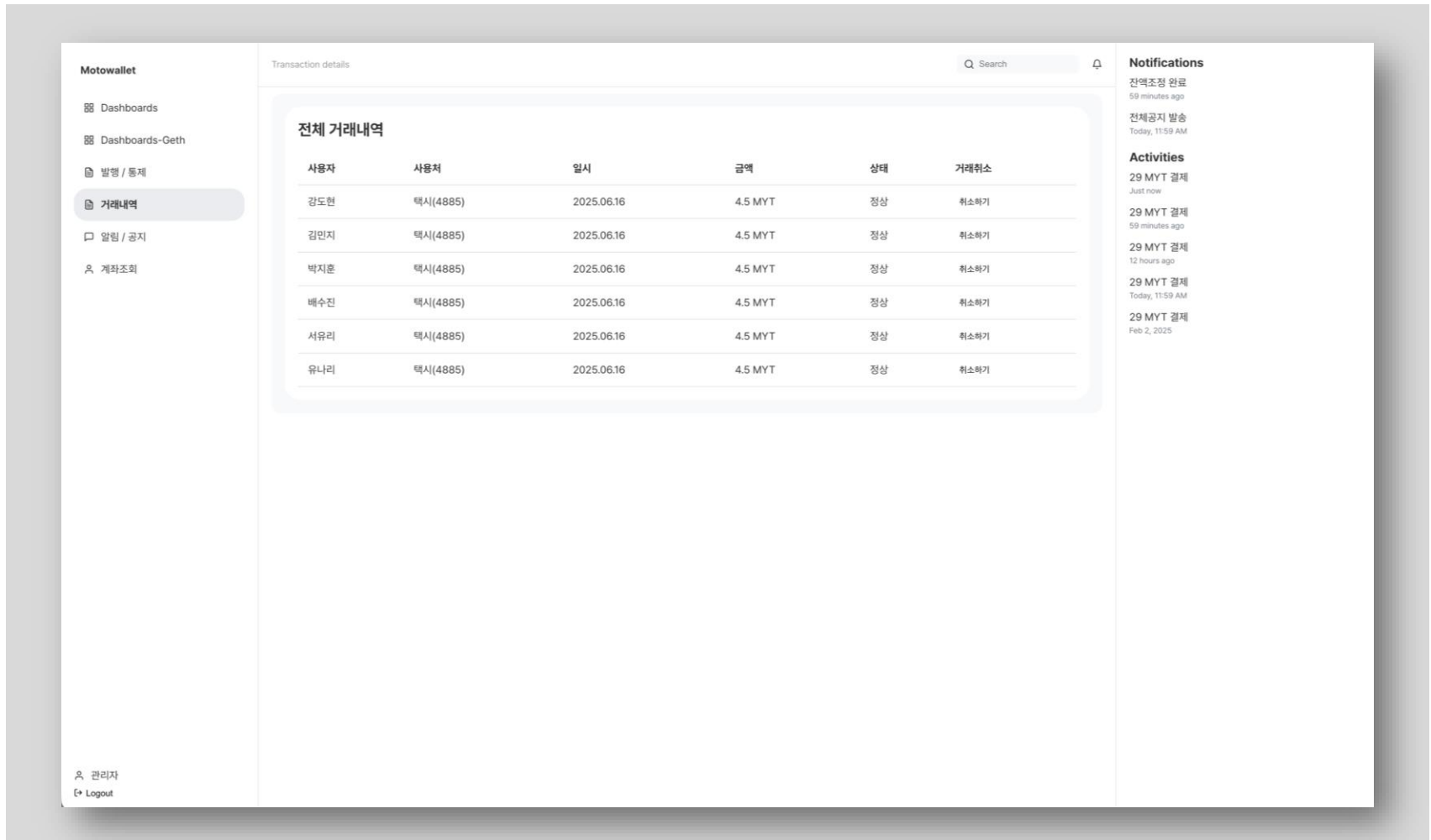
Feb 2, 2025

관리자

Logout

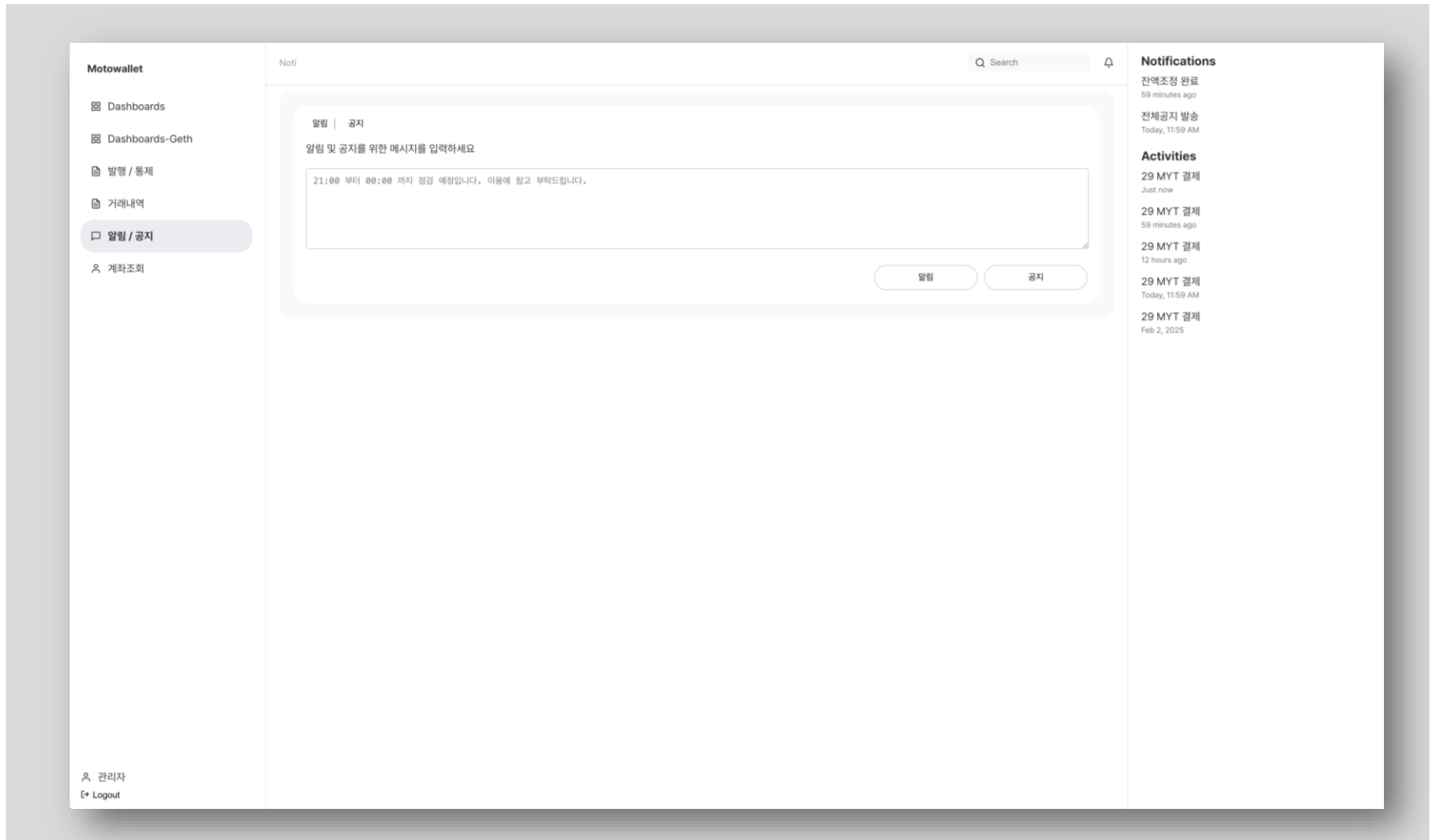
▲ 토큰 발행 관리 화면

# | 화면 설계서 - 관리자 인터페이스(sw)



▲ 전체 거래내역 확인

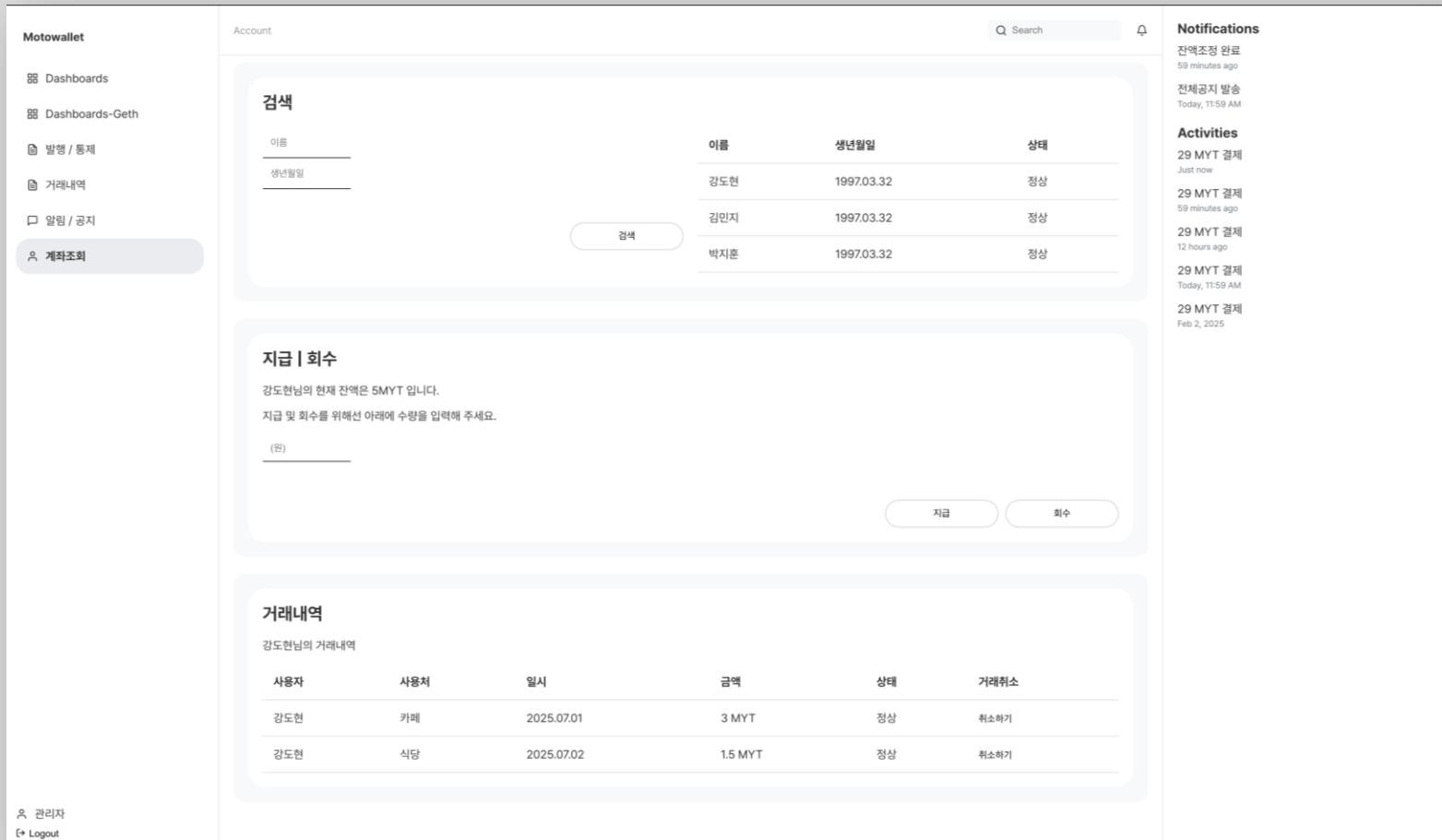
## | 화면 설계서 - 관리자 인터페이스(sw)



▲ 사용자(앱)에게 알림 및 공지사항 등록

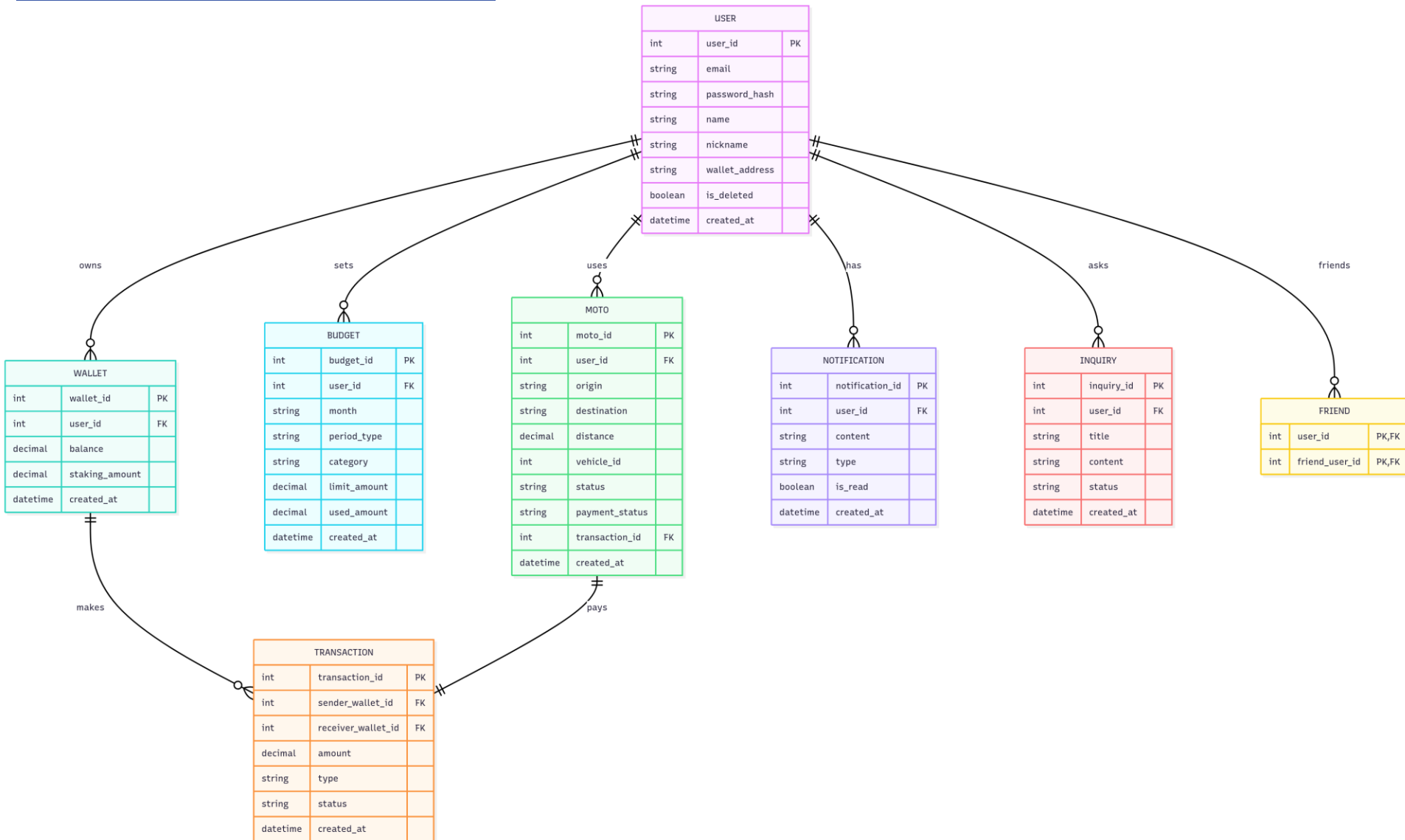


# | 화면 설계서 - 관리자 인터페이스(sw)



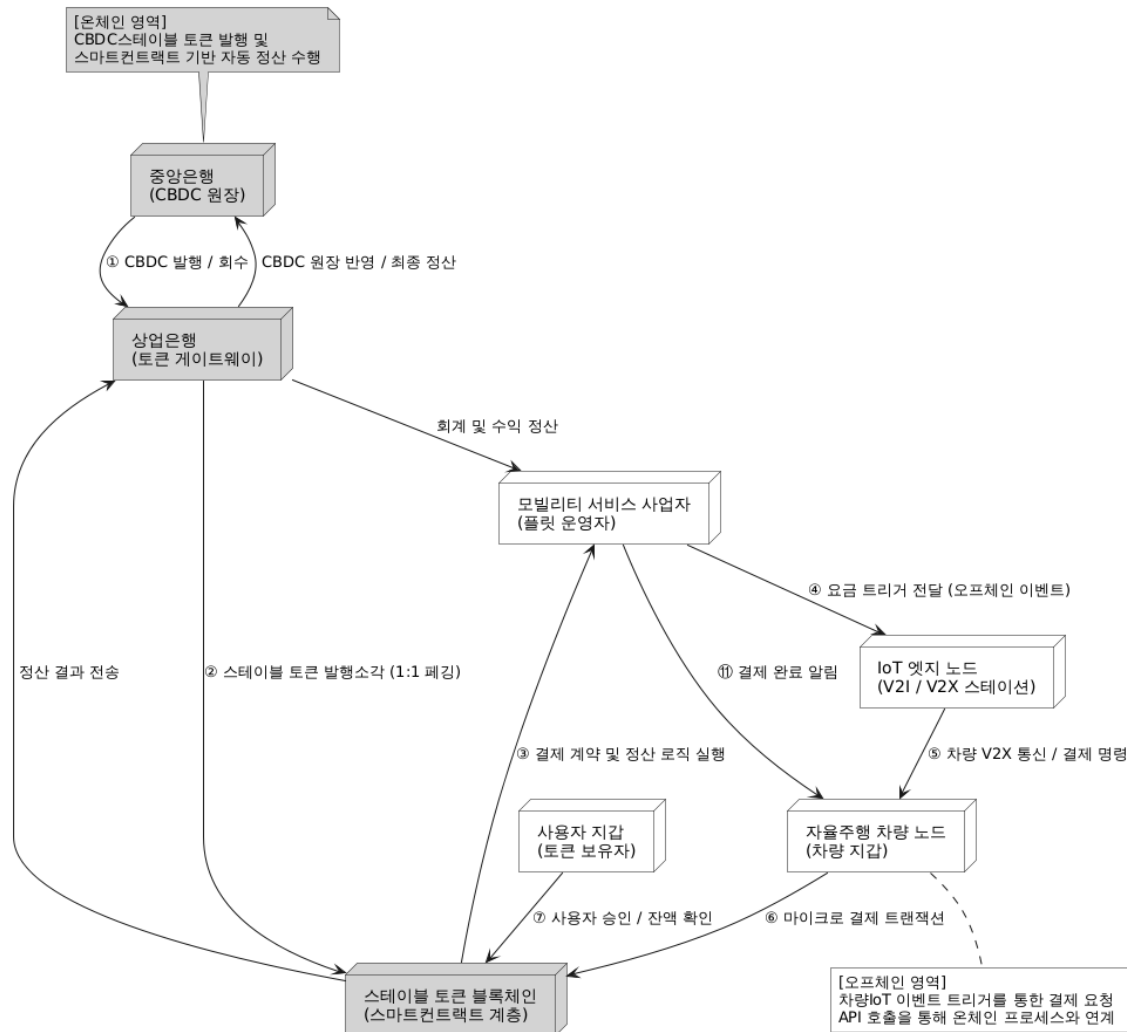
## ▲ 사용자별 지급계정 조회 및 거래 관리

# | 엔티티 관계도 - ERD

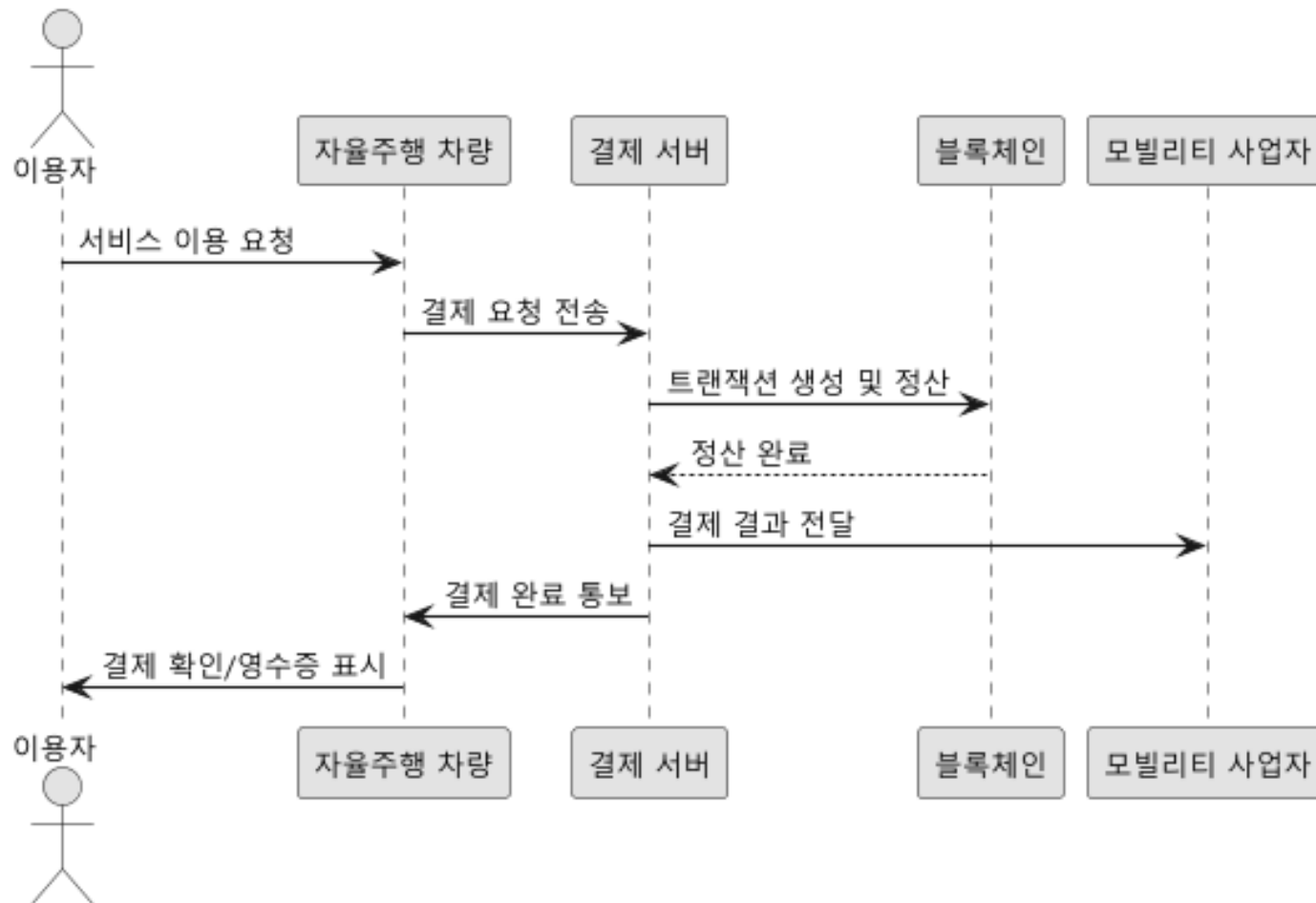


# | 시스템 구성도

## CBDC-스테이블 토큰 하이브리드 결제 프레임워크



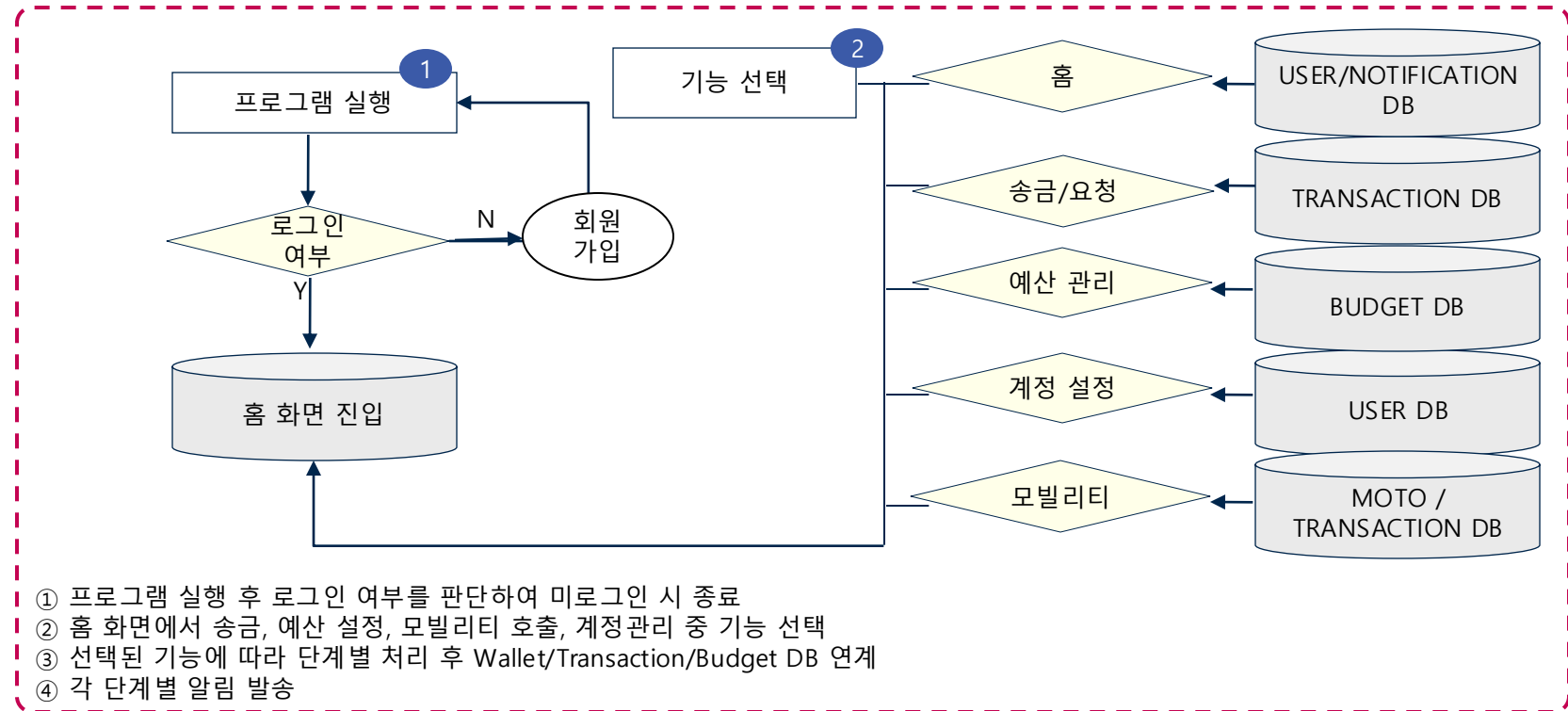
## | 시퀀스 다이어그램



# | 기능 처리도(기능 흐름도)

|         |                                                           |        |       |     |                  |
|---------|-----------------------------------------------------------|--------|-------|-----|------------------|
| 프로그램 ID | WALLET_MOTO_FNC_V1                                        | 프로그램 명 | MOLET | 작성일 | 2025 . 10 . 10 . |
| 개요      | 사용자 지갑 관리, 송금 요청, 예산 설정, 모빌리티 호출·QR 결제를 하나로 연계 처리하는 통합 모듈 |        |       |     |                  |
| 기능 흐름도  |                                                           |        |       |     |                  |

## <MOLET 기능 흐름도>



## | 핵심 소스코드(1) – 자율주행 객체 인식

```

import cv2

if __name__ == '__main__':
    img = cv2.imread('car_driving.jpg')
    (b,g,r) = img[100,100]
    print(b,g,r)
    i=j=0
    for j in range(1,255):
        img[i,j] = (0,0,0)
        for j in range(1,255):
            img[i,j] = (0,0,0)
    while True :
        cv2.imshow("frame",img)
        action = cv2.waitKey(10) & 0xFF
        if action == ord('q') or action == 113:
            break
    img.release()
    cv2.destroyAllWindows()

```



## | 핵심 소스코드(2) – 앱 HOME

```
Future<String> _getKakaoAuthCode() async {
  final redirectUri = _kakaoRedirectUri;
  if (await isKakaoTalkInstalled()) {
    return await AuthCodeClient.instance.authorizeWithTalk(redirectUri: redirectUri);
  }
  return await AuthCodeClient.instance.authorize(redirectUri: redirectUri);
}

Future<Map<String, dynamic>> _loginViaBackend(String authCode) async {
  final dio = Dio(BaseOptions(baseUrl: _apiBase));
  final resp = await dio.post('/api/auth/login', queryParameters: {'code':
authCode});
  return resp.data as Map<String, dynamic>;
}
```

- 카카오톡을 통해 **사용자 인증 코드 (authCode)**를 발급받고, 이를 서버 (/api/auth/login)에 전달하여 **JWT 토큰**을 발급받는 과정을 담당함.

```
Future<void> _storeTokens(Map<String, dynamic> data) async {
  final access = data['jwt'] ?? data['accessToken'];
  final refresh = data['refreshToken'];
  const storage = FlutterSecureStorage();
  if (access != null) await storage.write(key: 'jwt', value: access);
  if (refresh != null) await storage.write(key: 'refresh', value:
refresh);
}
```

- 서버에서 받은 **AccessToken**과 **RefreshToken**을 추출하여, FlutterSecureStorage에 저장함으로써 **로그인 상태를 유지하고 재사용할 수 있도록** 관리함.

## | 핵심 소스코드(3) – 앱 WALLET

```
static final ValueNotifier<int> balanceVN = ValueNotifier(100);
static final ValueNotifier<List<Map<String,dynamic>>> txVN = ValueNotifier(const []);

static Future<bool> pay(int amount,{String title='모빌리티 결제'}) async {
  if (balanceVN.value < amount) return false;
  balanceVN.value -= amount;
  txVN.value = [{ 'title':title, 'amount':-
amount, 'createdAt':DateTime.now().toIso8601String().substring(0,10)}, ...txVN.value];
  // SharedPreferences에 저장
  return true;
}
```

```
// WalletPage
onPressed: () async {
  final res = await Navigator.push(..., MaterialPageRoute(builder: (_) => const TransferPage()));
  if (res?['event']=='transfer_completed') {
    await WalletService.transfer(res['amount'], toName: res['toName']);
  }
}

// TransferPage
onTap: () async {
  final amount = await showAmountSheet(context);
  final ok = await showDialog(...);
  if (ok==true) Navigator.pop(context,
{'event':'transfer_completed', 'amount':amount, 'toName':f['name']});
}
```

- 사용자의 **현재 잔액(balanceVN)**을 확인하여 결제 금액이 충분할 때만 진행함.
- 결제 성공 시 잔액에서 금액을 차감하고, **거래 내역(title, amount, createdAt)**을 기록하여 이후에도 확인 가능하게 관리함.
- **WalletPage**에서는 사용자가 버튼을 누르면 TransferPage로 이동하고, 송금이 완료되면 결과 이벤트 (transfer\_completed)를 받아 거래를 실행함.
- **TransferPage**에서는 사용자가 송금 금액을 입력하고 확인 시, **송금 완료 이벤트(amount, toName)**를 WalletPage로 반환함.



## | 핵심 소스코드(4) – 앱 MOBILITY

```

void _onMapTap(LatLng p) async {
  if (_origin == null) { _origin = p; _originName = await _reverseGeocode(p); }
  else if (_dest == null) { _dest = p; _destName = await _reverseGeocode(p); }
  else { _origin = p; _dest = null; _originName = await _reverseGeocode(p); }
  setState(() {});
}

Future<String> _reverseGeocode(LatLng p) async {
  final r = await _dio.get('/v2/local/geo/coord2address.json',
    queryParameters: {'x': p.longitude, 'y': p.latitude});
  final docs = (r.data?['documents'] as List?) ?? [];
  return docs.isEmpty ? '${p.latitude}, ${p.longitude}'
    : (docs.first['road_address']?['building_name'] ??
      docs.first['address']?['address_name'] ?? '').toString();
}

```

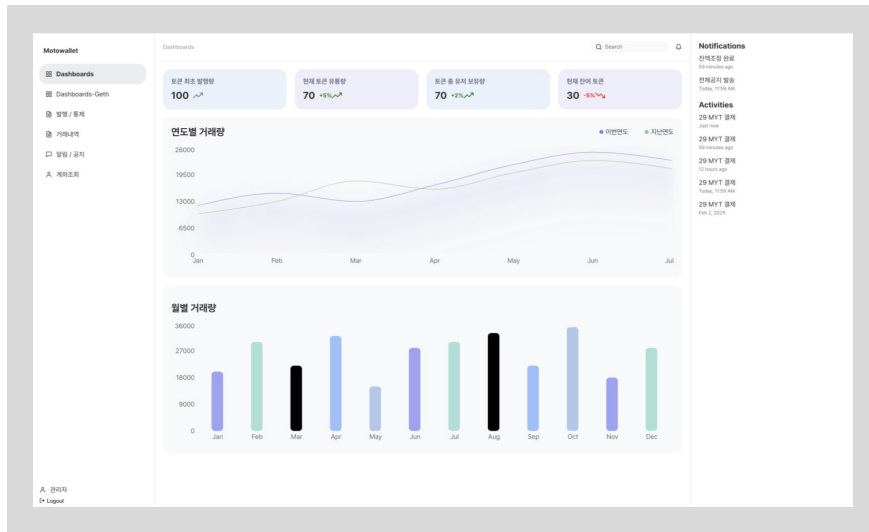
**\_onMapTap()** 함수에서 사용자가 지도에서 탭한 좌표를 **출발지(\_origin)**와 **도착지(\_dest)**로 설정하고, 해당 좌표를 **\_reverseGeocode()** 함수로 전달하여 카카오 지도 API에서 도로명·건물명·주소(\_originName, \_destName)를 조회한 뒤 화면에 표시함.

## | 핵심 소스코드(4) – 앱 MOBILITY

```
FilledButton.icon(
  icon: const Icon(Icons.directions_car_filled),
  label: const Text('차량 호출'),
  onPressed: (_origin!=null && _dest!=null) ? () {
    final km = _distKm(_origin!, _dest!);
    final est = _fare(km); // 추정 요금
    final session = TripSession(
      tripId: DateTime.now().millisecondsSinceEpoch.toString(),
      wsUrl: const String.fromEnvironment('VEHICLE_WS_URL'),
      demo: false,
    );
    Navigator.push(context, MaterialPageRoute(
      builder: (_) => WaitingPage(session: session, origin: _origin!, dest: _dest!, estimate:
est)));
  } : null,
);
```

FilledButton.icon을 통해 차량 호출 버튼을 만들고, onPressed 시 **\_origin · \_dest 좌표를 기반으로 거리(\_distKm)**와 **요금(\_fare)**을 계산하여 TripSession을 생성한 뒤, WaitingPage로 이동해 호출 결과를 표시함.

# | 핵심 소스코드(5) – 관리자 웹 대시보드



```
app.get('/api/dashboard/cards', async (req, res) => {
  try {
    const query = `
      SELECT
        (SELECT COUNT(*) FROM payments WHERE status = 'completed') AS payment_count,
        (SELECT SUM(amount) FROM transactions) AS transaction_amount,
        (SELECT COUNT(*) FROM transfers) AS transfer_count,
        (SELECT CONCAT(ROUND((SUM(CASE WHEN is_abnormal = 1 THEN 1 ELSE 0 END) /
          COUNT(*)) * 100, 2), '%') FROM transactions) AS abnormal_transactions
    `;
    const [rows] = await pool.query(query);

    if (rows.length > 0) {
      res.json(rows[0]);
    } else {
      res.status(404).json({ message: "Dashboard data not found." });
    }
  } catch (error) {
    console.error('Error fetching dashboard card data:', error);
    res.status(500).json({ error: 'Internal Server Error', message: error.message });
  }
});
```

```
<div className="dashboard-header">
  <h1>Dashboard</h1>
</div>
<div className="dashboard-cards">
  <div className="card">
    <div className="card-header">
      <h3>결제건수</h3>
      <FiBox className="card-icon" />
    </div>
    <p className="card-value">{cardData.payment_count.toLocaleString()}</p>
  </div>
  <div className="card">
    <div className="card-header">
      <h3>거래대금</h3>
      <FiActivity className="card-icon" />
    </div>
    <p className="card-value">{cardData.transaction_amount.toLocaleString()}</p>
  </div>
</div>
```

## | 핵심 소스코드(6) – 코인 발행 및 관리

```

public WalletResponseDTO createWallet(Long memberId) {
    // 1. 사용자 존재 확인
    Member member = memberRepository.findById(memberId)
        .orElseThrow(() -> new IllegalArgumentException("사용자 없음"));

    // 2. GetNewAccount 요청 JSON 생성
    String json = """
        {
            "jsonrpc": "2.0",
            "method": "personal_newAccount",
            "params": ["*"],
            "id": 1
        }
        """;

    // 3. HTTP 요청 헤더 및 본문 구성
    HttpHeaders headers = new HttpHeaders();
    headers.setContentType(MediaType.APPLICATION_JSON);

    HttpEntity<String> request = new HttpEntity<>(json, headers);
    RestTemplate restTemplate = restTemplateBuilder.build();

    // 4. GetNewAccount 요청
    ResponseEntity<String> response = restTemplate.postForEntity(GETH_URL, request, String.class);

    // 5. 응답 처리
    Map<String, Object> responseBody = response.getBody();
    if (responseBody == null || !responseBody.containsKey("result")) {
        throw new RuntimeException("GetNewAccount 실패: " + responseBody);
    }

    String address = (String) responseBody.get("result");
    if (address == null || !address.startsWith("0x")) {
        throw new RuntimeException("GetNewAccount 실패: " + responseBody);
    }

    // 6. Wallet 객체 생성 및 저장 (privateKey는 필요하지 않음)
    Wallet wallet = Wallet.builder()
        .address(address)
        .privateKey(null) // Geth API 제공된 키를 사용하지 않음
        .number(1)
        .build();

    try {
        // 4-1. 가스리미트를 ETH로 설정
        sendInitialEth(address, new BigInteger("1000000000000000000000")); // 1 ETH
        // 4-2. 토큰 지급
        WalletInitTokenDTO walletInitTokenDTO = sendInitToken(address, BigInteger.valueOf(100)); // 100 MYT

        WalletTransaction walletTransaction = WalletTransaction.builder()
            .toAddress(address)
            .tokenName("MYT")
            .amount(walletInitTokenDTO.getAmount())
            .fromAddress(FUNDER_ADDRESS)
            .fromName("FUNDER")
            .token(walletInitTokenDTO.getToken())
            .build();

        walletRepository.save(wallet);
        walletTransactionRepository.save(walletTransaction);

        return new WalletResponseDTO(wallet, List.of(walletTransaction));
    } catch (Exception e) {
        log.error("CreateWallet 실패: {}", e);
        throw new RuntimeException("CreateWallet 실패: " + e);
    }
}

```

```

private void sendInitialEth(String to, BigInteger weiAmount) throws Exception {
    Web3j web3 = Web3j.build(new HttpService(GETH_URL));

    try {
        // 1) 펀더 개인키로 서명할 준비
        Credentials creds = Credentials.create(FUNDER_PRIVATE_KEY);
        log.info("Funder addr used by app = {}", creds.getAddress());

        // receipt 기다리지 않는 전송 (필요하면 PollingTransactionReceiptProcessor로 바꾸세요)
        FastRawTransactionManager txManager =
            new FastRawTransactionManager(web3, creds, CHAIN_ID, new NoOpProcessor(web3));

        // 2) 가스 가격 (설정이 0이면 RPC 값 사용)
        BigInteger gasPrice = (GAS_PRICE_WEI != null && GAS_PRICE_WEI.signum() > 0)
            ? GAS_PRICE_WEI
            : web3.ethGasPrice().send().getGasPrice();

        // 3) ETH 전송은 보통 21,000 가스
        BigInteger gasLimit = BigInteger.valueOf(21_000);

        // 4) 서명 + 전송
        EthSendTransaction sent = txManager.sendTransaction(
            gasPrice,
            gasLimit,
            to,
            weiAmount,
            null // data 없음
            null // 보내는 ETH(wei)
        );

        if (sent.hasError()) {
            throw new RuntimeException("send ETH error: " + sent.getError().getMessage());
        }
        log.info("ETH 전송 성공 to={}, amountWei={}, tx={}", to, weiAmount,
            sent.getTransactionHash());
    } finally {
        web3.shutdown();
    }
}

```

- **CreateWallet()** - 사용자의 지갑을 처음 생성하는 부분
- geth에 계정 생성을 요청하고, 만들어진 사용자 지갑 주소로 **sendInitialEth()**와 **sendInitialToken()**을 통해 초기 토큰 값 **100MYT**를 발급함

## | 핵심 소스코드(7) – 카메라 제어

```
class MyPiCamera():  
    def __init__(self, width, height):  
        self.cap = Picamera2()  
        self.config = self.cap.create_video_configuration(  
            main={"format": "RGB888", "size": (width, height)})  
        self.cap.configure(self.config)  
        self.cap.start()  
  
    def read(self):  
        frame = self.cap.capture_array()  
        return True, cv2.rotate(frame, cv2.ROTATE_180)
```

라즈베리파이 카메라(PiCamera2)에서 실시간 영상을 받아오는 부분이다.  
차선 인식과 모터 제어를 위한 기본 영상 입력 장치로 활용된다.

## | 핵심 소스코드(8) – 모터 제어

```
class Motor:
    def __init__(self, PWMA, AIN1, AIN2, PWMB, BIN1, BIN2):
        GPIO.setup(PWMA, GPIO.OUT)
        GPIO.setup(PWMB, GPIO.OUT)
        self.L_Motor = GPIO.PWM(PWMA, 500)
        self.R_Motor = GPIO.PWM(PWMB, 500)

    def drive(self, speed=50, turn=0.0):
        left_speed = speed * (1 + min(turn, 0))
        right_speed = speed * (1 - max(turn, 0))
        self.L_Motor.ChangeDutyCycle(left_speed)
        self.R_Motor.ChangeDutyCycle(right_speed)

    def stop(self):
        self.L_Motor.ChangeDutyCycle(0)
        self.R_Motor.ChangeDutyCycle(0)
```

좌·우 모터 속도를 **독립적으로 제어**해 직진과 회전을 구현한다.  
**turn 값**(스티어링 비율)에 따라 좌우 속도가 다르게 동작한다.

## | 핵심 소스코드(9) – 차선인식 알고리즘

```
def lane_detection(image, lower_bound, upper_bound,
prev_steering=0.0):
    img_hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
    mask = cv2.inRange(img_hsv, lower_bound, upper_bound)
    dist_transform = cv2.distanceTransform(mask, cv2.DIST_L2, 5)
    _, max_val, _, max_loc = cv2.minMaxLoc(dist_transform)

    center_x = max_loc[0]
    error = (image.shape[1] // 2) - center_x
    steering = 0.7 * prev_steering + 0.3 * (error /
(image.shape[1]/4))
    return steering
```

HSV 색공간에서 녹색 차선을 인식하고, 중심 좌표를 계산해 차량의 스티어링 값을 산출한다.  
이 값은 모터 제어에 반영된다.

## | 핵심 소스코드(10) -LED 제어 로직

```
def lane_def led_control(mode):  
    if mode == 1: # 깜빡임  
        threading.Thread(target=blink_led).start()  
    elif mode == 2: # 항상 켜기  
        GPIO.output(16, GPIO.HIGH)  
        GPIO.output(26, GPIO.HIGH)  
    elif mode == 5: # 끄기  
        GPIO.output(16, GPIO.LOW)  
        GPIO.output(26, GPIO.LOW)  
  
def blink_led():  
    while True:  
        GPIO.output(16, GPIO.HIGH)  
        GPIO.output(26, GPIO.HIGH)  
        time.sleep(0.5)  
        GPIO.output(16, GPIO.LOW)  
        GPIO.output(26, GPIO.LOW)  
        time.sleep(0.5)
```

LED는 차량 상태를 나타내는 역할을 한다.

- **깜빡임**: 호출 표시
- **항상 켜짐**: 정상 주행(빈 차)
- **꺼짐**: 빈 차가 아님



## | 핵심 소스코드(11) – 신호등 정지 로직

```
def traffic_light_detection(image):
    img_hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

    # 빨간색 범위 (2개 영역으로 분리: 저채도, 고채도)
    red_lower1 = np.array([0, 100, 100])
    red_upper1 = np.array([10, 255, 255])
    red_lower2 = np.array([160, 100, 100])
    red_upper2 = np.array([179, 255, 255])

    mask1 = cv2.inRange(img_hsv, red_lower1, red_upper1)
    mask2 = cv2.inRange(img_hsv, red_lower2, red_upper2)
    red_mask = cv2.bitwise_or(mask1, mask2)

    # 빨간색 픽셀 면적 계산
    red_area = cv2.countNonZero(red_mask)

    # 특정 픽셀 이상 감지되면 '빨간불'
    if red_area > 500:
        return True
    return False
```

- 카메라 영상(BGR)을 HSV 색공간으로 변환
- 빨간색은 두 영역(0-10°, 160-179°)으로 나눠 검출
- 마스크 합성 후 픽셀 면적이 일정 값(500픽셀 이상)이면 **빨간불로 판단** → 정지

## | 핵심 소스코드(12) : LiDAR 센서 입력

```
from rplidar import RPLidar

# 포트는 실제 연결된 장치에 따라 다름 (/dev/ttyUSB0 또는 /dev/ttyAMA0 등)
LIDAR_PORT = "/dev/ttyUSB0"
lidar = RPLidar(LIDAR_PORT)

def get_lidar_data():
    """
    RPLIDAR에서 각도별 거리 데이터를 읽어 dict 형태로 반환
    {angle: distance}
    """
    lidar_data = {}
    for scan in lidar.iter_scans(max_buf_meas=500):
        for (_, angle, distance) in scan:
            angle_int = int(angle) % 360
            # distance는 mm 단위 → cm 변환
            lidar_data[angle_int] = distance / 10.0
    return lidar_data
```

## | 핵심 소스코드(13) 정류장 도착 감지 로직

```
import cv2, pytesseract

def bus_stop_detection(image):
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY) # 흑백 변환
    _, threshed = cv2.threshold(gray, 150, 255,
    cv2.THRESH_BINARY) # 이진화
    text = pytesseract.image_to_string(threshed, lang='kor+eng')
    # OCR 인식

    # 정류장 이름과 번호를 동시에 확인
    if "서울역" in text or "133" in text:
        return "서울역(133)"
    elif "종각역" in text or "131" in text:
        return "종각역(131)"
    else:
        return None
```

- 이미지 전처리 : 컬러 → 흑백 → 이진화
- OCR 처리 : pytesseract 로 글자 추출
- 정류장 이름 비교 : "서울역", "종각역" 포함 시 → True
- 정류장 번호 비교: "131", "133" 포함 시 → True
- 결과 : 정류장 도착 시 차량 정지 및 대기

## | 핵심 소스코드(14) : 부저 제어 로직

```
import RPi.GPIO as GPIO
import time

# 부저 핀 설정
BUZZER_PIN = 20 # GPIO 핀 번호

# GPIO 초기화
GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)
GPIO.setup(BUZZER_PIN, GPIO.OUT)

def buzzer_beep(freq=2000, duration=0.2):
    """주파수 freq(Hz)로 duration(초)만큼 부저 울리기"""
    pwm = GPIO.PWM(BUZZER_PIN, freq)
    pwm.start(50) # 듀티사이클 50%
    time.sleep(duration)
    pwm.stop()

def payment_failed_alert():
    """결제 실패 시 부저를 3회 울리는 함수"""
    for _ in range(3):
        buzzer_beep(freq=2000, duration=0.2) # 0.2초 간 부저
        time.sleep(0.2) # 0.2초 간격

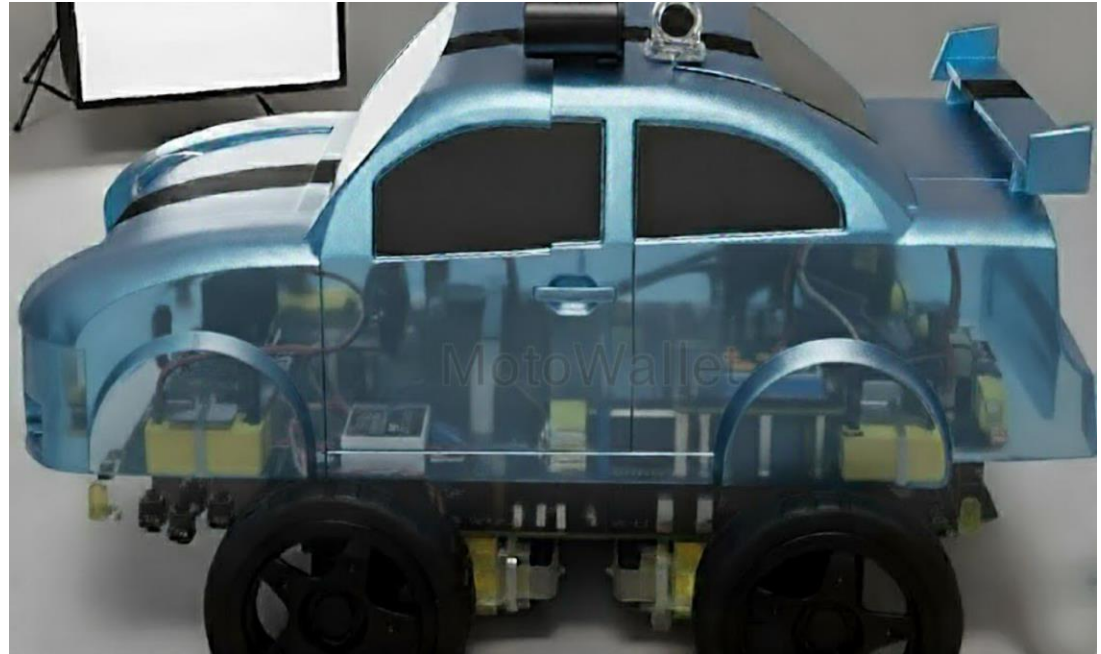
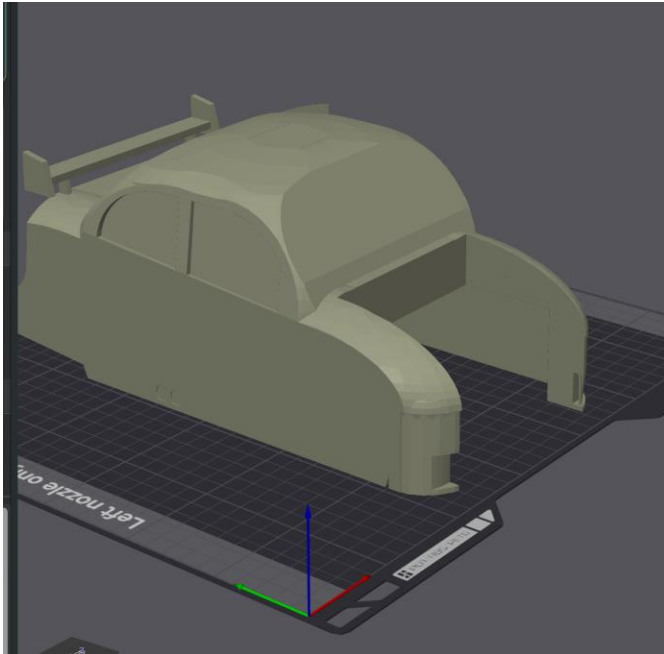
if __name__ == "__main__":
    try:
        # 결제 실패 이벤트 시 호출
        print("결제 실패! 부저 울림")
        payment_failed_alert()
    except KeyboardInterrupt:
        pass
    finally:
        GPIO.output(BUZZER_PIN, GPIO.LOW)
        GPIO.cleanup()
```

- **PWM 사용** → 스피커형 부저에 맞춰 주파수로 소리 발생
- **짧게 3회 울림** → 결제 실패 알림용으로 적합
- **주파수/길이 조정 가능** → buzzer\_beep(freq, duration) 매개변수 조정 가능
- **GPIO 안전 정리** → 종료 시 GPIO.cleanup()

## | 참조-개발 환경 및 설명

| 구분            |            | 상세 내용                                                    |
|---------------|------------|----------------------------------------------------------|
| S/W<br>개발 환경  | OS         | macOS, windows                                           |
|               | 개발 환경(IDE) | Visual Studio Code, IntelliJ IDEA, Android studio        |
|               | 배포 환경      | AWS, Docker                                              |
|               | 개발 언어      | Flutter, Javascript, Java, Dart, Solidity                |
|               | 기타 사항      | Node.js, Spring boot, MySQL                              |
| H/W<br>구성 장비  | 디바이스       | Raspberry Pi 4 Model B, Pi Racer, Blender , Bambu Studio |
|               | 센서         | Raspberry Pi Camera Module V2, LiDAR                     |
|               | 통신         | Wi-Fi, WebSocket                                         |
|               | 언어         | Python, C++                                              |
|               | 기타 사항      | OpenCV, Tensorflow                                       |
| 프로젝트<br>관리 환경 | 형상 관리      | Github                                                   |
|               | 의사소통 관리    | Discord, Notion                                          |
|               | 기타 사항      | 카카오톡 단체 채팅방을 통한 지속적인 피드백 관리, 멘토 님과의 소통                   |

## | 참조-H/W 기능 실사 사진



- ✓ 차체에는 **카메라(Camera)**, **LiDAR**, **LED**, **Buzzer**가 모두 장착되어 있으며, 라즈베리파이와 LiDAR 센서의 안정성을 확보하기 위해 상자형 구조물을 활용하였다. 이를 통해 외부 충격이나 진동으로부터 주요 부품을 보호하고, 장비의 내구성을 강화할 수 있도록 설계하였다.
- ✓ 또한 차량은 4개의 바퀴가 각각 DC 모터와 직접 연결되어 있어, 독립적인 구동 제어가 가능하다. 이를 통해 주행 안정성과 다양한 주행 알고리즘 실험을 지원할 수 있다.

# | 참조-동영상 시나리오

## 동영상 시나리오

### 1. 로그인(Molet)

- 사용자는 소셜로그인 기반으로 회원가입을 진행한다.
- 사용자는 앱을 실행하고 카카오 SSO를 통해 접속한다.
- 로그인에 성공하면 메인 화면으로 이동하며, 상단에는 현재 지갑 잔액과 최근 이용 내역이 표시된다.

### 2. 지갑 생성

- 첫 로그인 사용자는 지갑이 없으므로 "지갑 생성" 버튼을 눌러 지갑을 발급받는다.
- 생성 완료 후 초기 잔액은 100.00000 MYT 이다.

### 3. 자율주행 차량 호출

- 사용자는 출발지, 도착지를 입력하고 차량 호출 요청을 보낸다|
- (출발: 서울역 → 경유: 시청역 → 도착: 광화문 표지판 세워놓고 지도로 찍어놓기)
- 시스템은 현재 위치, 수요/공급 상태, 요금 정책을 고려해 예상 도착 시간(ETA)과 예상 요금을 보여준다. (MYT 로 요금 단위 수정)
- 사용자가 "차량 호출"을 누르면 배차가 진행되며, 배정된 차량 정보(차량 번호, 차량색, 배터리)표시된다.
- 출발지에 도착 → 사용자 탑승 시 QR 스캔

#### 3-1. 차량 호출

- 자율주행자동차 동작 프로세스(출발지, 목적지~~~~~)
- 앱에서 출발지, 목적지 설정 후 차량으로 전송 시 출발지로 출발.
- 출발지에 도착해 사용자의 탑승 시 qr 스캔 한다.
- 자동차로 탑승 메시지 수신시 출발(led꺼짐)
- 목적지로 출발한다.
- 신호등 발견시 정지한다.
- 주행중 장애물 발견시 정지한다.
- 부저를 통해 목적지에 도착했음을 알린다.
- 도착 후 qr 스캔을 통해 하차를 확인한다.(led 다시 켜짐) → 요금 지불 확인 후 미결제시 부저 알림
- 승객 하차 후 자율주행 모드로 트랙을 순회한다.

### 4. 차량 이동ing

- 실시간 처리를 통해 이동 거리, 이동 진행률 표시
- 사용자는 탑승 전, 차량 번호와 기사 정보를 앱과 실제 차량에서 한 번 더 대조한다.

### 5. 도착 및 하차

- 목적지에 도착하면 운행이 종료된다.
- 앱은 이동 거리, 소요 시간, 동선 요약, 기본 요금 및 추가 요금(심야/혼잡/돌게이트 등)을 계산하여 청구서를 생성한다.
- 사용자는 QR을 통해 청구서를 조회한다.
- 사용자는 청구서 내역을 확인하고 결제 단계로 이동한다.
- 결제 진행 후 결제내역을 확인한다.
- 결제 실패시 → 앱 : 잔액부족 표출, 차량: 부저음을 통한 알림

정상결제 보여주고, 잔액 부족시 친구에게 송금요청

### 7. 송금

1. 송금 요청 버튼을 통해 송금을 요청한다.
2. 금액을 입력하고 요청하기 버튼을 클릭한다.
3. 알림을 수신한 후 → 송금하기 → 금액입력 → 송금 확인 클릭
4. 그동안 잔액 부족 사용자는 결제 진행중에서 대기하다, 송금 완료시 결제 진행.

### 8. 남은 시간 관리자 화면

- 관리자는 대시보드에서 다음 정보를 실시간으로 확인한다.
  - 각 페이지별로 순회하며 자막을 통한 기능 설명
- 관리자는 특정 사용자를 선택해 세부 정보(최근 결제 내역, QR 결제 로그, 리스크 플래그)를 확인하고, 필요 시 수동 보정(환불/부분취소/재발행)을 수행할 수 있다.

## | 참조-프로젝트 관리

The screenshot shows a GitHub pull request for a commit titled "#1 chore: 개발환경 통합 작업 반영 #2". It includes a conversation tab with a comment from "이슈 번호" (Issue Number) and a commit history section showing the pull request was approved on August 15. The code diff shows changes to a Java file, specifically adding getters and setters for a DTO class.

▲ Git 관리

| Aa 회의 이름           | 📅 날짜          | ☰ 카테고리     | ☀️ 상태 |
|--------------------|---------------|------------|-------|
| 수행 계획서 작성          |               |            | 완료    |
| 사전 미팅              | 2025년 3월 13일  | 짧은 회의      | 완료    |
| kickoff 미팅 (04.13) | 2025년 4월 13일  | 비대면회의 멘토회의 | 완료    |
| 0506 미팅            | 2025년 5월 6일   | 대면회의 멘토회의  | 완료    |
| 0513 미팅            | 2025년 5월 13일  | 비대면회의      | 완료    |
| 0522 미팅            | 2025년 5월 22일  | 비대면회의      | 완료    |
| 0529 미팅            | 2025년 5월 29일  | 비대면회의      | 완료    |
| 중간평가 관련 회의         | 2025년 6월 29일  | 비대면회의 멘토회의 | 완료    |
| 0701 미팅            | 2025년 7월 1일   | 대면회의       | 완료    |
| 0710 미팅            | 2025년 8월 15일  | 대면회의       | 완료    |
| 0713 미팅            | 2025년 8월 6일   | 대면회의 멘토회의  | 완료    |
| 0801 미팅            | 2025년 8월 1일   | 비대면회의      | 완료    |
| 0810 미팅            | 2025년 8월 10일  | 비대면회의 멘토회의 | 완료    |
| 0815 미팅            | 2025년 8월 15일  | 대면회의       | 완료    |
| 0817 미팅            | 2025년 8월 15일  | 대면회의 멘토회의  | 완료    |
| 0821 미팅            | 2025년 8월 21일  | 대면회의       | 완료    |
| 0827 미팅            | 2025년 8월 27일  | 비대면회의      | 완료    |
| 0828 미팅            | 2025년 8월 28일  | 비대면회의 멘토회의 | 완료    |
| 1004 미팅            | 2025년 10월 4일  | 대면회의 멘토회의  | 완료    |
| 1007 미팅            | 2025년 10월 7일  | 비대면회의      | 완료    |
| 1019 미팅            | 2025년 10월 19일 | 비대면회의 멘토회의 | 완료    |
| 1025 미팅            | 2025년 10월 27일 | 비대면회의 멘토회의 | 완료    |
| 1026 미팅            | 2025년 10월 26일 | 대면회의 멘토회의  | 완료    |

▲ 별도 회의록 관리



# Thank you

